

CHAPTER 6

Linked Lists

LEARNING OBJECTIVE

A linked list is a collection of data elements called nodes in which the linear representation is given by links from one node to the next node. In this chapter, we are going to discuss different types of linked lists and the operations that can be performed on these lists.

6.1 INTRODUCTION

We have studied that an array is a linear collection of data elements in which the elements are stored in consecutive memory locations. While declaring arrays, we have to specify the size of the array, which will restrict the number of elements that the array can store. For example, if we declare an array as `int marks[10]`, then the array can store a maximum of 10 data elements but not more than that. But what if we are not sure of the number of elements in advance? Moreover, to make efficient use of memory, the elements must be stored randomly at any location rather than in consecutive locations. So, there must be a data structure that removes the restrictions on the maximum number of elements and the storage condition to write efficient programs.

Linked list is a data structure that is free from the aforementioned restrictions. A linked list does not store its elements in consecutive memory locations and the user can add any number of elements to it. However, unlike an array, a linked list does not allow random access of data. Elements in a linked list can be accessed only in a sequential manner. But like an array, insertions and deletions can be done at any point in the list in a constant time.

6.1.1 Basic Terminologies

A linked list, in simple terms, is a linear collection of data elements. These data elements are called *nodes*. Linked list is a data structure which in turn can be used to implement other data

structures. Thus, it acts as a building block to implement data structures such as stacks, queues, and their variations. A linked list can be perceived as a train or a sequence of nodes in which each node contains one or more data fields and a pointer to the next node.

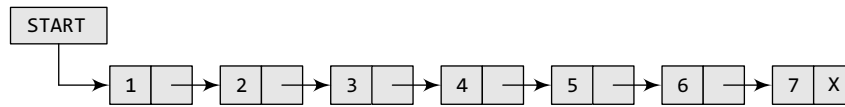


Figure 6.1 Simple linked list

In Fig. 6.1, we can see a linked list in which every node contains two parts, an integer and a pointer to the next node. The left part of the node which contains data may include a simple data type, an array, or a structure. The right part of the node contains a pointer to the next node (or address of the next node in sequence). The last node will have no next node connected to it, so it will store a special value called NULL. In Fig. 6.1, the NULL pointer is represented by x. While programming, we usually define NULL as -1. Hence, a NULL pointer denotes the end of the list. Since in a linked list, every node contains a pointer to another node which is of the same type, it is also called a *self-referential data type*.

Linked lists contain a pointer variable **START** that stores the address of the first node in the list. We can traverse the entire list using **START** which contains the address of the first node; the next part of the first node in turn stores the address of its succeeding node. Using this technique, the individual nodes of the list will form a chain of nodes. If **START** = NULL, then the linked list is empty and contains no nodes.

In C, we can implement a linked list using the following code:

```

struct node
{
    int data;
    struct node *next;
};
  
```

Note Linked lists provide an efficient way of storing related data and perform basic operations such as insertion, deletion, and updation of information at the cost of extra space required for storing address of the next node.

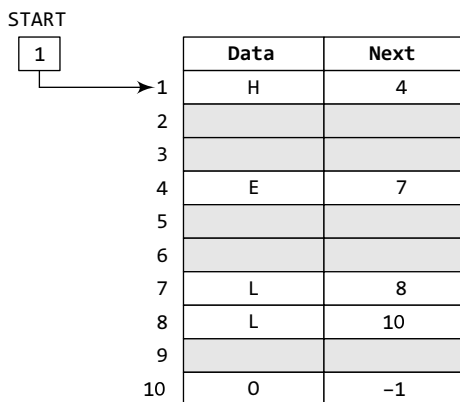


Figure 6.2 START pointing to the first element of the linked list in the memory

Let us see how a linked list is maintained in the memory. In order to form a linked list, we need a structure called *node* which has two fields, **DATA** and **NEXT**. **DATA** will store the information part and **NEXT** will store the address of the next node in sequence. Consider Fig. 6.2.

In the figure, we can see that the variable **START** is used to store the address of the first node. Here, in this example, **START** = 1, so the first data is stored at address 1, which is H. The corresponding **NEXT** stores the address of the next node, which is 4. So, we will look at address 4 to fetch the next data item. The second data element obtained from address 4 is E. Again, we see the corresponding **NEXT** to go to the next node. From the entry in the **NEXT**, we get the next address, that is 7, and fetch L as the data. We repeat this procedure until we reach a position where the **NEXT** entry contains -1 or NULL, as this

would denote the end of the linked list. When we traverse DATA and NEXT in this manner, we finally see that the linked list in the above example stores characters that when put together form the word HELLO.

Note that Fig. 6.2 shows a chunk of memory locations which range from 1 to 10. The shaded portion contains data for other applications. Remember that the nodes of a linked list need not be in consecutive memory locations. In our example, the nodes for the linked list are stored at addresses 1, 4, 7, 8, and 10.

Let us take another example to see how two linked lists are maintained together in the computer's memory. For example, the students of Class XI of Science group are asked to choose between Biology and Computer Science. Now, we will maintain two linked lists, one for each subject. That is, the first linked list will contain the roll numbers of all the students who have opted for Biology and the second list will contain the roll numbers of students who have chosen Computer Science.

Now, look at Fig. 6.3, two different linked lists are simultaneously maintained in the memory. There is no ambiguity in traversing through the list because each list maintains a separate START pointer, which gives the address of the first node of their respective linked lists. The rest of the nodes are reached by looking at the value stored in the NEXT.

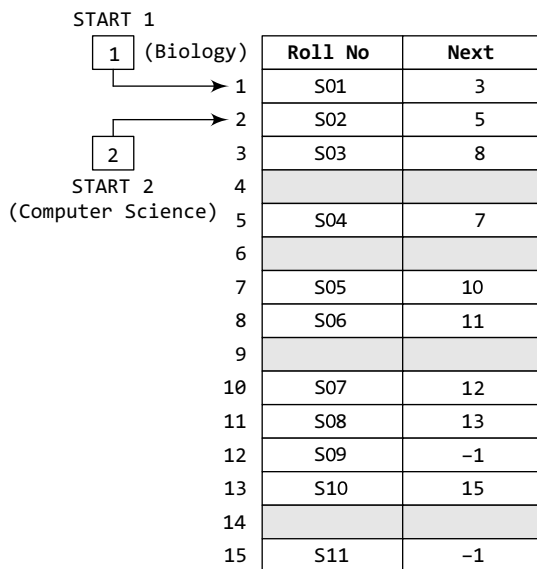


Figure 6.3 Two linked lists which are simultaneously maintained in the memory

By looking at the figure, we can conclude that roll numbers of the students who have opted for Biology are S01, S03, S06, S08, S10, and S11. Similarly, roll numbers of the students who chose Computer Science are S02, S04, S05, S07, and S09.

We have already said that the DATA part of a node may contain just a single data item, an array, or a structure. Let us take an example to see how a structure is maintained in a linked list that is stored in the memory.

Consider a scenario in which the roll number, name, aggregate, and grade of students are stored using linked lists. Now, we will see how the NEXT pointer is used to store the data alphabetically. This is shown in Fig. 6.4.

6.1.2 Linked Lists versus Arrays

Both arrays and linked lists are a linear collection of data elements. But unlike an array, a linked list does not store its nodes in consecutive memory locations. Another point of difference between an array and a linked list is that a linked list does not allow random access of data. Nodes in a linked list can be accessed only in a sequential manner. But like an array, insertions and deletions can be done at any point in the list in a constant time.

Another advantage of a linked list over an array is that we can add any number of elements in the list. This is not possible in case of an array. For example, if we declare an array as `int marks[20]`, then the array can store a maximum of 20 data elements only. There is no such restriction in case of a linked list.

	Roll No	Name	Aggregate	Grade	Next
1	S01	Ram	78	Distinction	6
2	S02	Shyam	64	First division	14
3					
4	S03	Mohit	89	Outstanding	17
5					
6	S04	Rohit	77	Distinction	2
7	S05	Varun	86	Outstanding	10
8	S06	Karan	65	First division	12
9					
10	S07	Veena	54	Second division	-1
11	S08	Meera	67	First division	4
12	S09	Krish	45	Third division	13
13	S10	Kusum	91	Outstanding	11
14	S11	Silky	72	First division	7
15					
16					
17	S12	Monica	75	Distinction	1
18	S13	Ashish	63	First division	19
19	S14	Gaurav	61	First division	8

START
18 →

Figure 6.4 Students' linked list

Thus, linked lists provide an efficient way of storing related data and performing basic operations such as insertion, deletion, and updation of information at the cost of extra space required for storing the address of next nodes.

6.1.3 Memory Allocation and De-allocation for a Linked List

We have seen how a linked list is represented in the memory. If we want to add a node to an already existing linked list in the memory, we first find free space in the memory and then use it to store the information. For example, consider the linked list shown in Fig. 6.5. The linked list contains the roll number of students, marks obtained by them in Biology, and finally a NEXT field which stores the address of the next node in sequence. Now, if a new student joins the class and is asked to appear for the same test that the other students had taken, then the new student's marks should also be recorded in the linked list. For this purpose, we find a free space and store the information there. In Fig. 6.5 the grey shaded portion shows free space, and thus we have 4 memory locations available. We can use any one of them to store our data. This is illustrated in Figs 6.5(a) and (b).

Now, the question is which part of the memory is available and which part is occupied? When we delete a node from a linked list, then who changes the status of the memory occupied by it from occupied to available? The answer is the operating system. Discussing the mechanism of how the operating system does all this is out of the scope of this book. So, in simple language, we can say that the computer does it on its own without any intervention from the user or the programmer. As a programmer, you just have to take care of the code to perform insertions and deletions in the list.

However, let us briefly discuss the basic concept behind it. The computer maintains a list of all free memory cells. This list of available space is called the *free pool*.

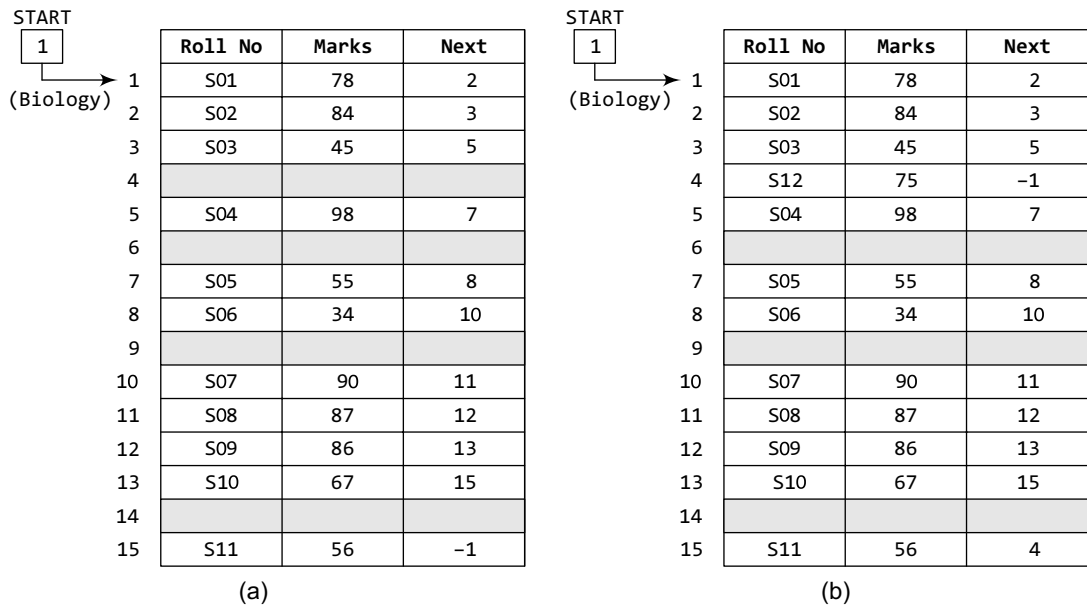


Figure 6.5 (a) Students' linked list and (b) linked list after the insertion of new student's record

We have seen that every linked list has a pointer variable *START* which stores the address of the first node of the list. Likewise, for the free pool (which is a linked list of all free memory cells), we have a pointer variable *AVAIL* which stores the address of the first free space. Let us revisit the memory representation of the linked list storing all the students' marks in Biology.

Now, when a new student's record has to be added, the memory address pointed by *AVAIL* will be taken and used to store the desired information. After the insertion, the next available free space's address will be stored in *AVAIL*. For example, in Fig. 6.6, when the first free memory space is utilized for inserting the new node, *AVAIL* will be set to contain address 6.

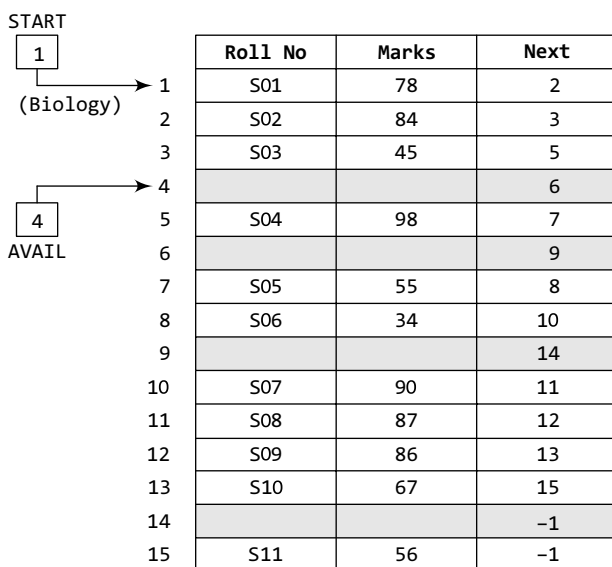


Figure 6.6 Linked list with *AVAIL* and *START* pointers

This was all about inserting a new node in an already existing linked list. Now, we will discuss deleting a node or the entire linked list. When we delete a particular node from an existing linked list or delete the entire linked list, the space occupied by it must be given back to the free pool so that the memory can be reused by some other program that needs memory space.

The operating system does this task of adding the freed memory to the free pool. The operating system will perform this operation whenever it finds the CPU idle or whenever the programs are falling short of memory space. The operating system scans through all the memory cells and marks those cells that are being used by some program. Then it collects all the cells which are not being used and adds

their address to the free pool, so that these cells can be reused by other programs. This process is called *garbage collection*.

There are different types of linked lists which we will discuss in the next section.

6.2 SINGLY LINKED LISTS

A singly linked list is the simplest type of linked list in which every node contains some data and a pointer to the next node of the same data type. By saying that the node contains a pointer to the next node, we mean that the node stores the address of the next node in sequence. A singly linked list allows traversal of data only in one way. Figure 6.7 shows a singly linked list.

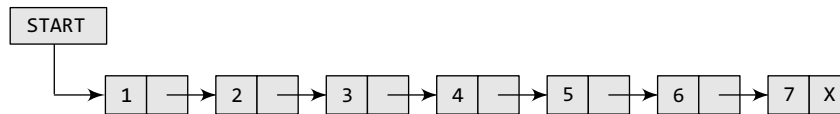


Figure 6.7 Singly linked list

6.2.1 Traversing a Linked List

Traversing a linked list means accessing the nodes of the list in order to perform some processing on them. Remember a linked list always contains a pointer variable `START` which stores the address of the first node of the list. End of the list is marked by storing `NULL` or `-1` in the `NEXT` field of the last node. For traversing the linked list, we also make use of another pointer variable `PTR` which points to the node that is currently being accessed. The algorithm to traverse a linked list is shown in Fig. 6.8.

In this algorithm, we first initialize `PTR` with the address of `START`. So now, `PTR` points to the first node of the linked list. Then in Step 2, a `while` loop is executed which is repeated till `PTR` processes the last node, that is until it encounters `NULL`. In Step 3, we apply the process (e.g., `print`) to the current node, that is, the node pointed by `PTR`. In Step 4, we move to the next node by making the `PTR` variable point to the node whose address is stored in the `NEXT` field.

```

Step 1: [INITIALIZE] SET PTR = START
Step 2: Repeat Steps 3 and 4 while PTR != NULL
Step 3:     Apply Process to PTR->DATA
Step 4:     SET PTR = PTR->NEXT
[END OF LOOP]
Step 5: EXIT

```

Figure 6.8 Algorithm for traversing a linked list

```

Step 1: [INITIALIZE] SET COUNT = 0
Step 2: [INITIALIZE] SET PTR = START
Step 3: Repeat Steps 4 and 5 while PTR != NULL
Step 4:     SET COUNT = COUNT + 1
Step 5:     SET PTR = PTR->NEXT
[END OF LOOP]
Step 6: Write COUNT
Step 7: EXIT

```

Figure 6.9 Algorithm to print the number of nodes in a linked list

Let us now write an algorithm to count the number of nodes in a linked list. To do this, we will traverse each and every node of the list and while traversing every individual node, we will increment the counter by 1. Once we reach `NULL`, that is, when all the nodes of the linked list have been traversed, the final value of the counter will be displayed. Figure 6.9 shows the algorithm to print the number of nodes in a linked list.

6.2.2 Searching for a Value in a Linked List

Searching a linked list means to find a particular element in the linked list. As already discussed, a linked list consists of nodes which are divided into two parts, the information part and the next part. So searching means finding whether a given value is present in the information part of the node or not. If it is present, the algorithm returns the address of the node that contains the value.

```

Step 1: [INITIALIZE] SET PTR = START
Step 2: Repeat Step 3 while PTR != NULL
Step 3:   IF VAL = PTR->DATA
           SET POS = PTR
           Go To Step 5
         ELSE
           SET PTR = PTR->NEXT
         [END OF IF]
       [END OF LOOP]
Step 4: SET POS = NULL
Step 5: EXIT

```

Figure 6.10 Algorithm to search a linked list

Figure 6.10 shows the algorithm to search a linked list.

In Step 1, we initialize the pointer variable PTR with START that contains the address of the first node. In Step 2, a while loop is executed which will compare every node's DATA with VAL for which the search is being made. If the search is successful, that is, VAL has been found, then the address of that node is stored in POS and the control jumps to the last statement of the algorithm. However, if the search is unsuccessful, POS is set to NULL which indicates that VAL is not present in the linked list.

Consider the linked list shown in Fig. 6.11. If we have VAL = 4, then the flow of the algorithm can be explained as shown in the figure.

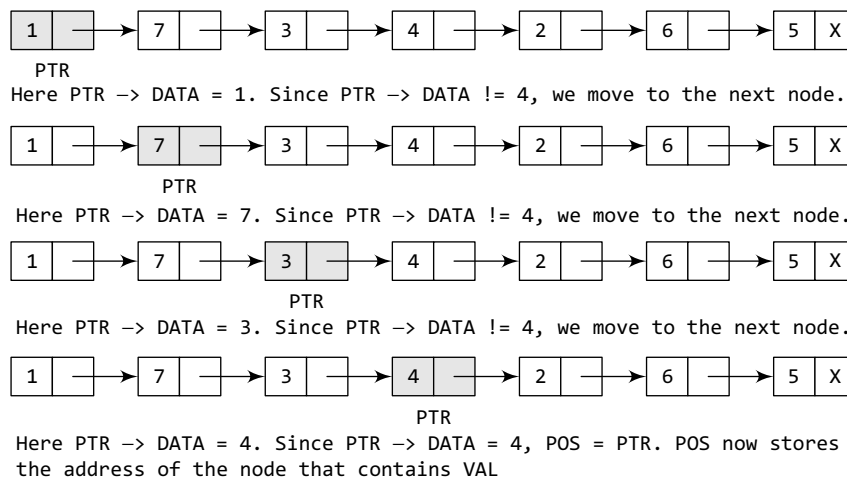


Figure 6.11 Searching a linked list

6.2.3 Inserting a New Node in a Linked List

In this section, we will see how a new node is added into an already existing linked list. We will take four cases and then see how insertion is done in each case.

Case 1: The new node is inserted at the beginning.

Case 2: The new node is inserted at the end.

Case 3: The new node is inserted after a given node.

Case 4: The new node is inserted before a given node.

Before we describe the algorithms to perform insertions in all these four cases, let us first discuss an important term called **OVERFLOW**. Overflow is a condition that occurs when AVAIL = NULL or no free memory cell is present in the system. When this condition occurs, the program must give an appropriate message.

Inserting a Node at the Beginning of a Linked List

Consider the linked list shown in Fig. 6.12. Suppose we want to add a new node with data 9 and add it as the first node of the list. Then the following changes will be done in the linked list.

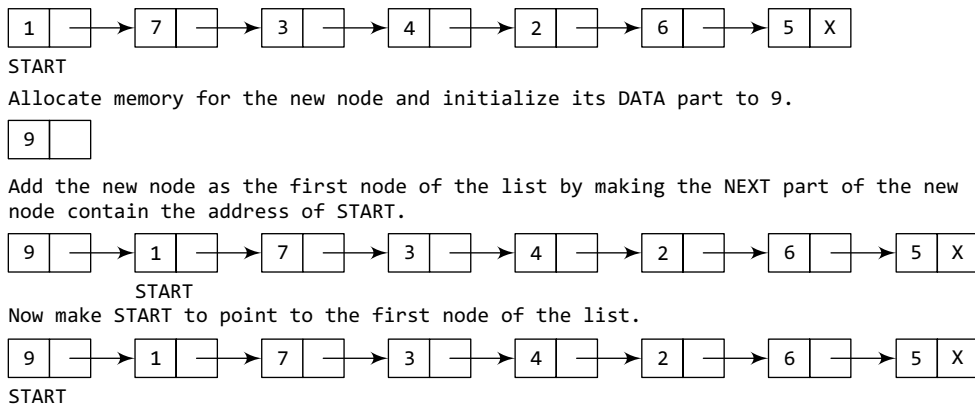


Figure 6.12 Inserting an element at the beginning of a linked list

```

Step 1: IF AVAIL = NULL
        Write OVERFLOW
        Go to Step 7
    [END OF IF]
Step 2: SET NEW_NODE = AVAIL
Step 3: SET AVAIL = AVAIL -> NEXT
Step 4: SET NEW_NODE -> DATA = VAL
Step 5: SET NEW_NODE -> NEXT = START
Step 6: SET START = NEW_NODE
Step 7: EXIT

```

Figure 6.13 Algorithm to insert a new node at the beginning

Figure 6.13 shows the algorithm to insert a new node at the beginning of a linked list. In Step 1, we first check whether memory is available for the new node. If the free memory has exhausted, then an **OVERFLOW** message is printed. Otherwise, if a free memory cell is available, then we allocate space for the new node. Set its DATA part with the given VAL and the NEXT part is initialized with the address of the first node of the list, which is stored in START. Now, since the new node is added as the first node of the list, it will now be known as the START node, that is, the START pointer variable will now hold the address of the NEW_NODE. Note the following two steps:

```

Step 2: SET NEW_NODE = AVAIL
Step 3: SET AVAIL = AVAIL -> NEXT

```

These steps allocate memory for the new node. In C, there are functions like `malloc()`, `alloc`, and `calloc()` which automatically do the memory allocation on behalf of the user.

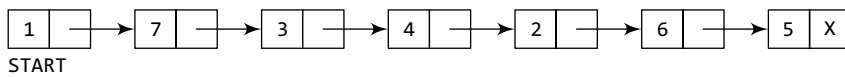
Inserting a Node at the End of a Linked List

Consider the linked list shown in Fig. 6.14. Suppose we want to add a new node with data 9 as the last node of the list. Then the following changes will be done in the linked list.

Figure 6.15 shows the algorithm to insert a new node at the end of a linked list. In Step 6, we take a pointer variable PTR and initialize it with START. That is, PTR now points to the first node of the linked list. In the while loop, we traverse through the linked list to reach the last node. Once we reach the last node, in Step 9, we change the NEXT pointer of the last node to store the address of the new node. Remember that the NEXT field of the new node contains NULL, which signifies the end of the linked list.

Inserting a Node After a Given Node in a Linked List

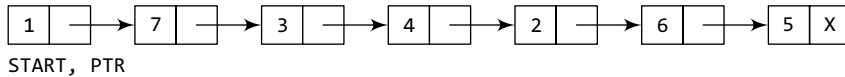
Consider the linked list shown in Fig. 6.17. Suppose we want to add a new node with value 9 after the node containing data 3. Before discussing the changes that will be done in the linked list, let us first look at the algorithm shown in Fig. 6.16.



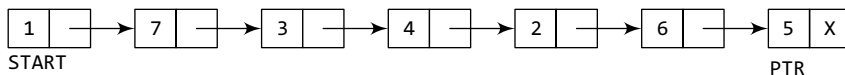
Allocate memory for the new node and initialize its DATA part to 9 and NEXT part to NULL.



Take a pointer variable PTR which points to START.



Move PTR so that it points to the last node of the list.



Add the new node after the node pointed by PTR. This is done by storing the address of the new node in the NEXT part of PTR.

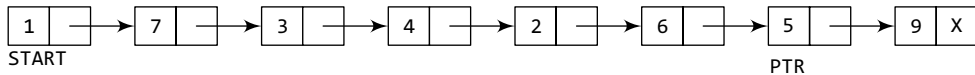


Figure 6.14 Inserting an element at the end of a linked list

```

Step 1: IF AVAIL = NULL
        Write OVERFLOW
        Go to Step 10
    [END OF IF]
Step 2: SET NEW_NODE = AVAIL
Step 3: SET AVAIL = AVAIL -> NEXT
Step 4: SET NEW_NODE -> DATA = VAL
Step 5: SET NEW_NODE -> NEXT = NULL
Step 6: SET PTR = START
Step 7: Repeat Step 8 while PTR -> NEXT != NULL
Step 8:     SET PTR = PTR -> NEXT
    [END OF LOOP]
Step 9: SET PTR -> NEXT = NEW_NODE
Step 10: EXIT
  
```

Figure 6.15 Algorithm to insert a new node at the end

```

Step 1: IF AVAIL = NULL
        Write OVERFLOW
        Go to Step 12
    [END OF IF]
Step 2: SET NEW_NODE = AVAIL
Step 3: SET AVAIL = AVAIL -> NEXT
Step 4: SET NEW_NODE -> DATA = VAL
Step 5: SET PTR = START
Step 6: SET PREPTR = PTR
Step 7: Repeat Steps 8 and 9 while PREPTR -> DATA
        != NUM
Step 8:     SET PREPTR = PTR
Step 9:     SET PTR = PTR -> NEXT
    [END OF LOOP]
Step 10: PREPTR -> NEXT = NEW_NODE
Step 11: SET NEW_NODE -> NEXT = PTR
Step 12: EXIT
  
```

Figure 6.16 Algorithm to insert a new node after a node that has value NUM

In Step 5, we take a pointer variable PTR and initialize it with START. That is, PTR now points to the first node of the linked list. Then we take another pointer variable PREPTR which will be used to store the address of the node preceding PTR. Initially, PREPTR is initialized to PTR. So now, PTR, PREPTR, and START are all pointing to the first node of the linked list.

In the while loop, we traverse through the linked list to reach the node that has its value equal to NUM. We need to reach this node because the new node will be inserted after this node. Once we reach this node, in Steps 10 and 11, we change the NEXT pointers in such a way that new node is inserted after the desired node.

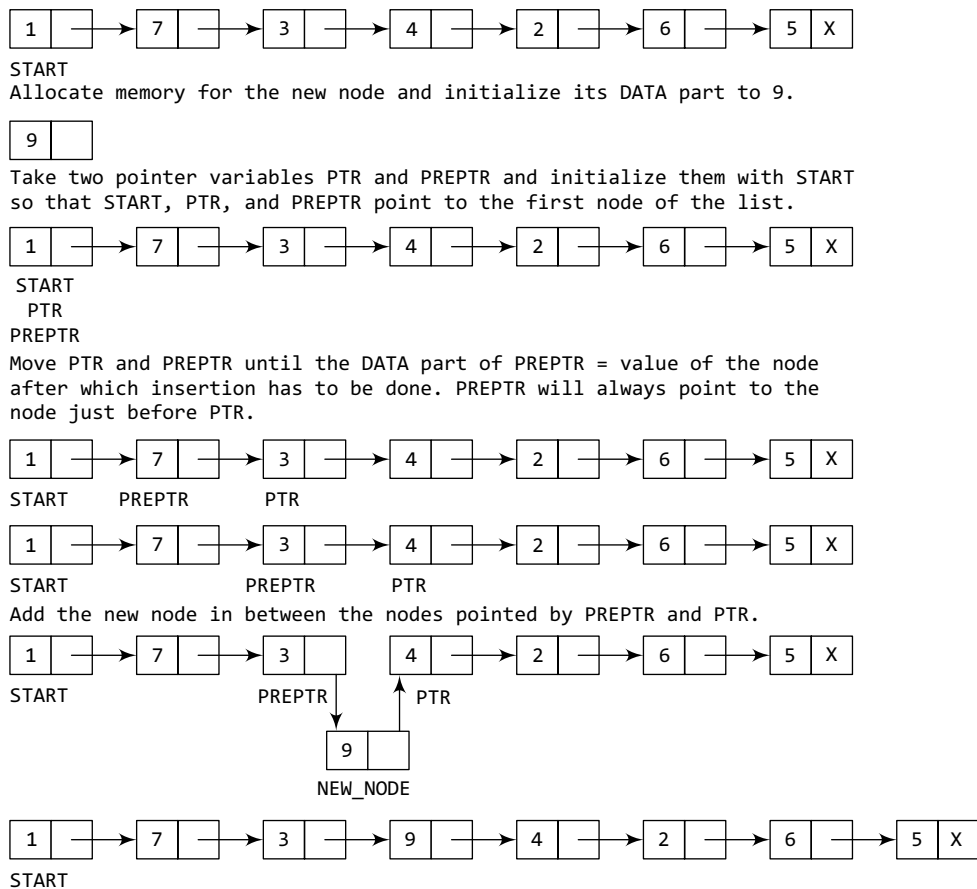


Figure 6.17 Inserting an element after a given node in a linked list

Inserting a Node Before a Given Node in a Linked List

Consider the linked list shown in Fig. 6.19. Suppose we want to add a new node with value 9 before

the node containing 3. Before discussing the changes that will be done in the linked list, let us first look at the algorithm shown in Fig. 6.18.

In Step 5, we take a pointer variable PTR and initialize it with START. That is, PTR now points to the first node of the linked list. Then, we take another pointer variable PREPTR and initialize it with PTR. So now, PTR, PREPTR, and START are all pointing to the first node of the linked list.

In the while loop, we traverse through the linked list to reach the node that has its value equal to NUM. We need to reach this node because the new node will be inserted before this node. Once we reach

```

Step 1: IF AVAIL = NULL
        Write OVERFLOW
        Go to Step 12
    [END OF IF]
Step 2: SET NEW_NODE = AVAIL
Step 3: SET AVAIL = AVAIL->NEXT
Step 4: SET NEW_NODE->DATA = VAL
Step 5: SET PTR = START
Step 6: SET PREPTR = PTR
Step 7: Repeat Steps 8 and 9 while PTR->DATA != NUM
Step 8:     SET PREPTR = PTR
Step 9:     SET PTR = PTR->NEXT
    [END OF LOOP]
Step 10: PREPTR->NEXT = NEW_NODE
Step 11: SET NEW_NODE->NEXT = PTR
Step 12: EXIT
  
```

Figure 6.18 Algorithm to insert a new node before a node that has value NUM

this node, in Steps 10 and 11, we change the NEXT pointers in such a way that the new node is inserted before the desired node.

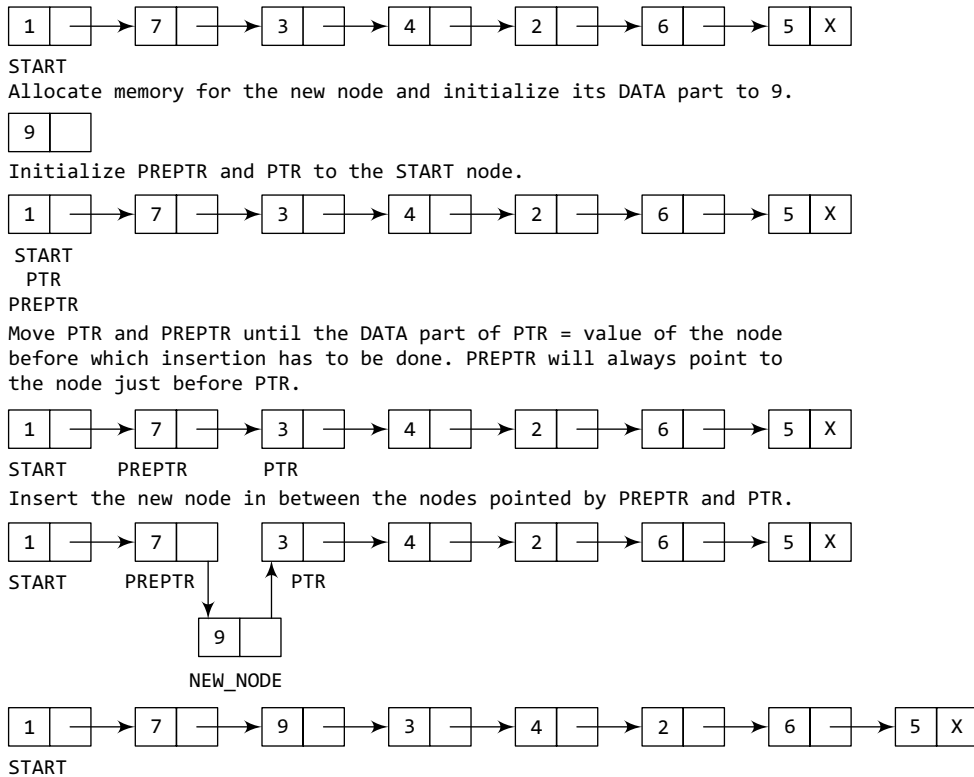


Figure 6.19 Inserting an element before a given node in a linked list

6.2.4 Deleting a Node from a Linked List

In this section, we will discuss how a node is deleted from an already existing linked list. We will consider three cases and then see how deletion is done in each case.

Case 1: The first node is deleted.

Case 2: The last node is deleted.

Case 3: The node after a given node is deleted.

Before we describe the algorithms in all these three cases, let us first discuss an important term called UNDERFLOW. Underflow is a condition that occurs when we try to delete a node from a linked list that is empty. This happens when $START = NULL$ or when there are no more nodes to delete. Note that when we delete a node from a linked list, we actually have to free the memory occupied by that node. The memory is returned to the free pool so that it can be used to store other programs and data. Whatever be the case of deletion, we always change the AVAIL pointer so that it points to the address that has been recently vacated.

Deleting the First Node from a Linked List

Consider the linked list in Fig. 6.20. When we want to delete a node from the beginning of the list, then the following changes will be done in the linked list.

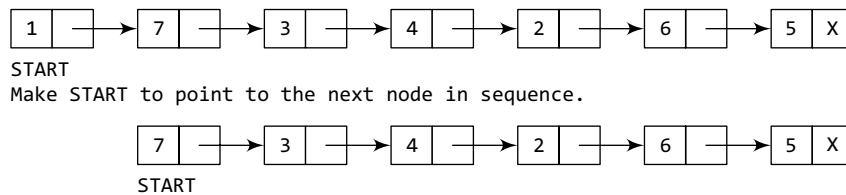


Figure 6.20 Deleting the first node of a linked list

Figure 6.21 shows the algorithm to delete the first node from a linked list. In Step 1, we check if the linked list exists or not. If $START = NULL$, then it signifies that there are no nodes in the list and the control is transferred to the last statement of the algorithm.

```

Step 1: IF  $START = NULL$ 
        Write UNDERFLOW
        Go to Step 5
    [END OF IF]
Step 2: SET  $PTR = START$ 
Step 3: SET  $START = START \rightarrow NEXT$ 
Step 4: FREE  $PTR$ 
Step 5: EXIT
    
```

Figure 6.21 Algorithm to delete the first node

However, if there are nodes in the linked list, then we use a pointer variable PTR that is set to point to the first node of the list. For this, we initialize PTR with $START$ that stores the address of the first node of the list. In Step 3, $START$ is made to point to the next node in sequence and finally the memory occupied by the node pointed by PTR (initially the first node of the list) is freed and returned to the free pool.

Deleting the Last Node from a Linked List

Consider the linked list shown in Fig. 6.22. Suppose we want to delete the last node from the linked list, then the following changes will be done in the linked list.

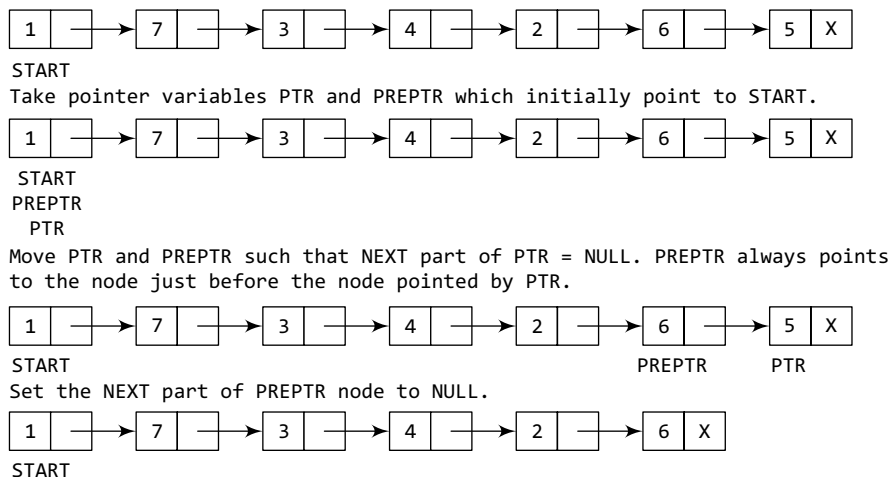


Figure 6.22 Deleting the last node of a linked list

Figure 6.23 shows the algorithm to delete the last node from a linked list. In Step 2, we take a pointer variable PTR and initialize it with $START$. That is, PTR now points to the first node of the linked list. In the $while$ loop, we take another pointer variable $PREPTR$ such that it always points to one node before the PTR . Once we reach the last node and the second last node, we set the $NEXT$ pointer of the second last node to $NULL$, so that it now becomes the (new) last node of the linked list. The memory of the previous last node is freed and returned back to the free pool.

```

Step 1: IF START = NULL
        Write UNDERFLOW
        Go to Step 8
    [END OF IF]
Step 2: SET PTR = START
Step 3: Repeat Steps 4 and 5 while PTR->NEXT != NULL
Step 4:     SET PREPTR = PTR
Step 5:     SET PTR = PTR->NEXT
    [END OF LOOP]
Step 6: SET PREPTR->NEXT = NULL
Step 7: FREE PTR
Step 8: EXIT

```

Figure 6.23 Algorithm to delete the last node

Deleting the Node After a Given Node in a Linked List

Consider the linked list shown in Fig. 6.24. Suppose we want to delete the node that succeeds the node which contains data value 4. Then the following changes will be done in the linked list.

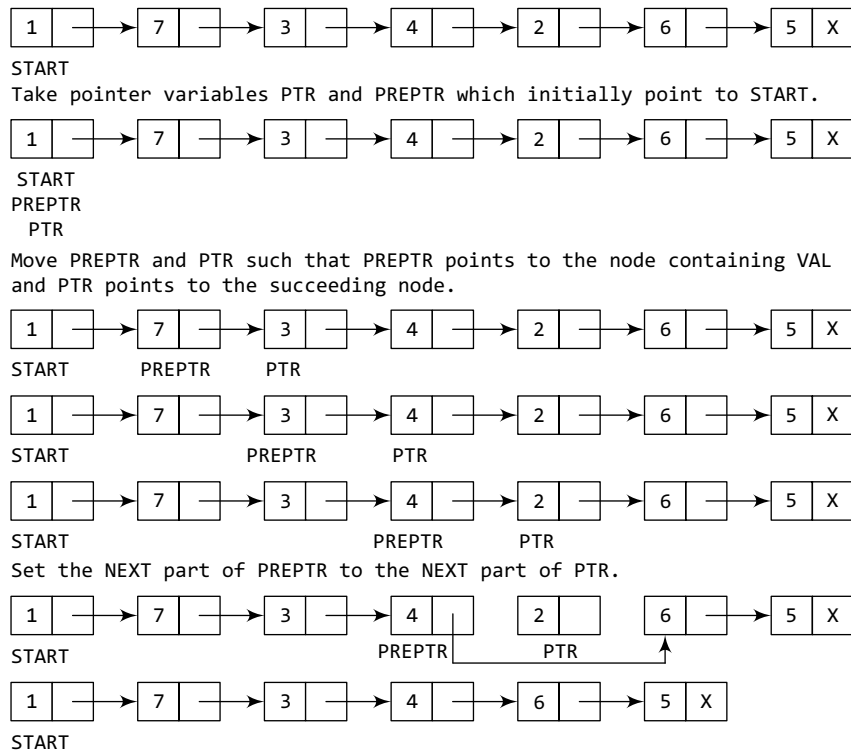


Figure 6.24 Deleting the node after a given node in a linked list

Figure 6.25 shows the algorithm to delete the node after a given node from a linked list. In Step 2, we take a pointer variable PTR and initialize it with START. That is, PTR now points to the first node of the linked list. In the while loop, we take another pointer variable PREPTR such that it always points to one node before the PTR. Once we reach the node containing VAL and the node succeeding it, we set the next pointer of the node containing VAL to the address contained in next field of the node succeeding it. The memory of the node succeeding the given node is freed and returned back to the free pool.

```

Step 1: IF START = NULL
        Write UNDERFLOW
        Go to Step 10
    [END OF IF]
Step 2: SET PTR = START
Step 3: SET PREPTR = PTR
Step 4: Repeat Steps 5 and 6 while PREPTR->DATA != NUM
Step 5:     SET PREPTR = PTR
Step 6:     SET PTR = PTR->NEXT
    [END OF LOOP]
Step 7: SET TEMP = PTR
Step 8: SET PREPTR->NEXT = PTR->NEXT
Step 9: FREE TEMP
Step 10: EXIT

```

Figure 6.25 Algorithm to delete the node after a given node

PROGRAMMING EXAMPLE

1. Write a program to create a linked list and perform insertions and deletions of all cases. Write functions to sort and finally delete the entire list at once.

```

#include <stdio.h>
#include <stdlib.h>
#include <conio.h>
#include <malloc.h>
struct node
{
    int data;
    struct node *next;
};
struct node *start = NULL;
struct node *create_ll(struct node *);
struct node *display(struct node *);
struct node *insert_beg(struct node *);
struct node *insert_end(struct node *);
struct node *insert_before(struct node *);
struct node *insert_after(struct node *);
struct node *delete_beg(struct node *);
struct node *delete_end(struct node *);
struct node *delete_node(struct node *);
struct node *delete_after(struct node *);
struct node *delete_list(struct node *);
struct node *sort_list(struct node *);

int main(int argc, char *argv[]) {
    int option;
    do
    {
        printf("\n\n *****MAIN MENU *****");
        printf("\n 1: Create a list");
        printf("\n 2: Display the list");
        printf("\n 3: Add a node at the beginning");
        printf("\n 4: Add a node at the end");
        printf("\n 5: Add a node before a given node");
        printf("\n 6: Add a node after a given node");
        printf("\n 7: Delete a node from the beginning");
    }

```

```

        printf("\n 8: Delete a node from the end");
        printf("\n 9: Delete a given node");
        printf("\n 10: Delete a node after a given node");
        printf("\n 11: Delete the entire list");
        printf("\n 12: Sort the list");
        printf("\n 13: EXIT");
        printf("\n\n Enter your option : ");
        scanf("%d", &option);
        switch(option)
        {
        case 1: start = create_ll(start);
                printf("\n LINKED LIST CREATED");
                break;
        case 2: start = display(start);
                break;
        case 3: start = insert_beg(start);
                break;
        case 4: start = insert_end(start);
                break;
        case 5: start = insert_before(start);
                break;
        case 6: start = insert_after(start);
                break;
        case 7: start = delete_beg(start);
                break;
        case 8: start = delete_end(start);
                break;
        case 9: start = delete_node(start);
                break;
        case 10: start = delete_after(start);
                break;
        case 11: start = delete_list(start);
                printf("\n LINKED LIST DELETED");
                break;
        case 12: start = sort_list(start);
                break;
        }
    }while(option !=13);
    getch();
    return 0;
}

struct node *create_ll(struct node *start)
{
    struct node *new_node, *ptr;
    int num;
    printf("\n Enter -1 to end");
    printf("\n Enter the data : ");
    scanf("%d", &num);
    while(num!=-1)
    {
        new_node = (struct node*)malloc(sizeof(struct node));
        new_node -> data=num;
        if(start==NULL)
        {
            new_node -> next = NULL;
            start = new_node;
        }
        else
        {
            ptr=start;

```

```

        while(ptr->next!=NULL)
            ptr=ptr->next;
        ptr->next = new_node;
        new_node->next=NULL;
    }
    printf("\n Enter the data : ");
    scanf("%d", &num);
}
return start;
}
struct node *display(struct node *start)
{
    struct node *ptr;
    ptr = start;
    while(ptr != NULL)
    {
        printf("\t %d", ptr -> data);
        ptr = ptr -> next;
    }
    return start;
}
struct node *insert_beg(struct node *start)
{
    struct node *new_node;
    int num;
    printf("\n Enter the data : ");
    scanf("%d", &num);
    new_node = (struct node *)malloc(sizeof(struct node));
    new_node -> data = num;
    new_node -> next = start;
    start = new_node;
    return start;
}
struct node *insert_end(struct node *start)
{
    struct node *ptr, *new_node;
    int num;
    printf("\n Enter the data : ");
    scanf("%d", &num);
    new_node = (struct node *)malloc(sizeof(struct node));
    new_node -> data = num;
    new_node -> next = NULL;
    ptr = start;
    while(ptr -> next != NULL)
        ptr = ptr -> next;
    ptr -> next = new_node;
    return start;
}
struct node *insert_before(struct node *start)
{
    struct node *new_node, *ptr, *preptr;
    int num, val;
    printf("\n Enter the data : ");
    scanf("%d", &num);
    printf("\n Enter the value before which the data has to be inserted : ");
    scanf("%d", &val);
    new_node = (struct node *)malloc(sizeof(struct node));
    new_node -> data = num;
    ptr = start;
    while(ptr -> data != val)
    {

```



```
        preptr = ptr;
        ptr = ptr -> next;
    }
    preptr -> next = new_node;
    new_node -> next = ptr;
    return start;
}

struct node *insert_after(struct node *start)
{
    struct node *new_node, *ptr, *preptr;
    int num, val;
    printf("\n Enter the data : ");
    scanf("%d", &num);
    printf("\n Enter the value after which the data has to be inserted : ");
    scanf("%d", &val);
    new_node = (struct node *)malloc(sizeof(struct node));
    new_node -> data = num;
    ptr = start;
    preptr = ptr;
    while(preptr -> data != val)
    {
        preptr = ptr;
        ptr = ptr -> next;
    }
    preptr -> next = new_node;
    new_node -> next = ptr;
    return start;
}

struct node *delete_beg(struct node *start)
{
    struct node *ptr;
    ptr = start;
    start = start -> next;
    free(ptr);
    return start;
}

struct node *delete_end(struct node *start)
{
    struct node *ptr, *preptr;
    ptr = start;
    while(ptr -> next != NULL)
    {
        preptr = ptr;
        ptr = ptr -> next;
    }
    preptr -> next = NULL;
    free(ptr);
    return start;
}

struct node *delete_node(struct node *start)
{
    struct node *ptr, *preptr;
    int val;
    printf("\n Enter the value of the node which has to be deleted : ");
    scanf("%d", &val);
    ptr = start;
    if(ptr -> data == val)
    {
        start = delete_beg(start);
        return start;
    }
    else
    {

```

```

        while(ptr -> data != val)
        {
            preptr = ptr;
            ptr = ptr -> next;
        }
        preptr -> next = ptr -> next;
        free(ptr);
        return start;
    }
}
struct node *delete_after(struct node *start)
{
    struct node *ptr, *preptr;
    int val;
    printf("\n Enter the value after which the node has to deleted : ");
    scanf("%d", &val);
    ptr = start;
    preptr = ptr;
    while(preptr -> data != val)
    {
        preptr = ptr;
        ptr = ptr -> next;
    }
    preptr -> next=ptr -> next;
    free(ptr);
    return start;
}
struct node *delete_list(struct node *start)
{
    struct node *ptr; // Lines 252-254 were modified from original code to fix
    unresponsiveness in output window
    if(start!=NULL){
        ptr=start;
        while(ptr != NULL)
        {
            printf("\n %d is to be deleted next", ptr -> data);
            start = delete_beg(ptr);
            ptr = start;
        }
    }

    return start;
}
struct node *sort_list(struct node *start)
{
    struct node *ptr1, *ptr2;
    int temp;
    ptr1 = start;
    while(ptr1 -> next != NULL)
    {
        ptr2 = ptr1 -> next;
        while(ptr2 != NULL)
        {
            if(ptr1 -> data > ptr2 -> data)
            {
                temp = ptr1 -> data;
                ptr1 -> data = ptr2 -> data;
                ptr2 -> data = temp;
            }
            ptr2 = ptr2 -> next;
        }
        ptr1 = ptr1 -> next;
    }
}

```

```

    }
    return start; // Had to be added
}

```

Output

```

*****MAIN MENU *****
1: Create a list
2: Display the list
3: Add a node at the beginning
4: Add the node at the end
5: Add the node before a given node
6: Add the node after a given node
7: Delete a node from the beginning
8: Delete a node from the end
9: Delete a given node
10: Delete a node after a given node
11: Delete the entire list
12: Sort the list
13: Exit
Enter your option : 3
Enter your option : 73

```

6.3 CIRCULAR LINKED LISTS

In a circular linked list, the last node contains a pointer to the first node of the list. We can have a circular singly linked list as well as a circular doubly linked list. While traversing a circular linked list, we can begin at any node and traverse the list in any direction, forward or backward, until we reach the same node where we started. Thus, a circular linked list has no beginning and no ending. Figure 6.26 shows a circular linked list.

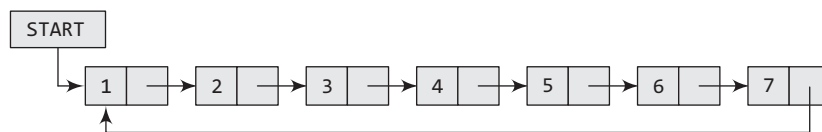


Figure 6.26 Circular linked list

The only downside of a circular linked list is the complexity of iteration. Note that there are no NULL values in the NEXT part of any of the nodes of list.

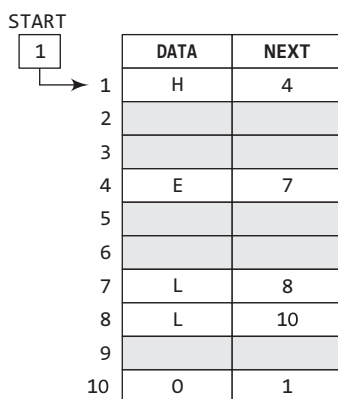


Figure 6.27 Memory representation of a circular linked list

Circular linked lists are widely used in operating systems for task maintenance. We will now discuss an example where a circular linked list is used. When we are surfing the Internet, we can use the Back button and the Forward button to move to the previous pages that we have already visited. How is this done? The answer is simple. A circular linked list is used to maintain the sequence of the Web pages visited. Traversing this circular linked list either in forward or backward direction helps to revisit the pages again using Back and Forward buttons. Actually, this is done using either the circular stack or the circular queue. We will read about circular queues in Chapter 8. Consider Fig. 6.27.

We can traverse the list until we find the NEXT entry that contains the address of the first node of the list. This denotes the end of the linked list, that is, the node that contains the address of the first node is actually

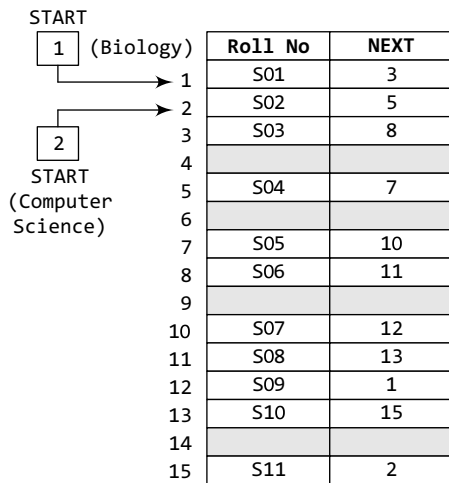


Figure 6.28 Memory representation of two circular linked lists stored in the memory

the last node of the list. When we traverse the DATA and NEXT in this manner, we will finally see that the linked list in Fig. 6.27 stores characters that when put together form the word HELLO.

Now, look at Fig. 6.28. Two different linked lists are simultaneously maintained in the memory. There is no ambiguity in traversing through the list because each list maintains a separate START pointer which gives the address of the first node of the respective linked list. The remaining nodes are reached by looking at the value stored in NEXT.

By looking at the figure, we can conclude that the roll numbers of the students who have opted for Biology are S01, S03, S06, S08, S10, and S11. Similarly, the roll numbers of the students who chose Computer Science are S02, S04, S05, S07, and S09.

6.3.1 Inserting a New Node in a Circular Linked List

In this section, we will see how a new node is added into an already existing linked list. We will take two cases and then see how insertion is done in each case.

Case 1: The new node is inserted at the beginning of the circular linked list.

Case 2: The new node is inserted at the end of the circular linked list.

Inserting a Node at the Beginning of a Circular Linked List

Consider the linked list shown in Fig. 6.29. Suppose we want to add a new node with data 9 as the first node of the list. Then the following changes will be done in the linked list.

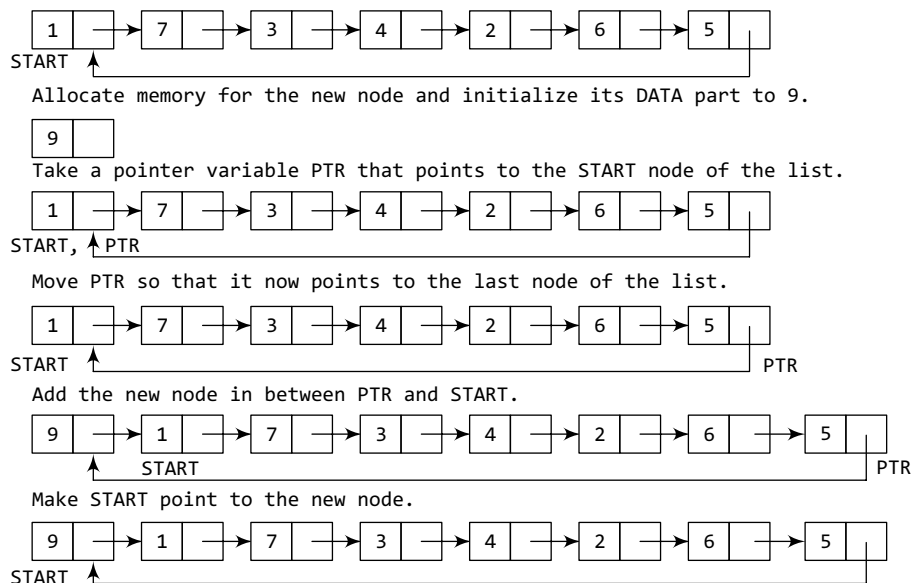


Figure 6.29 Inserting a new node at the beginning of a circular linked list

```

Step 1: IF AVAIL = NULL
        Write OVERFLOW
        Go to Step 11
    [END OF IF]
Step 2: SET NEW_NODE = AVAIL
Step 3: SET AVAIL = AVAIL -> NEXT
Step 4: SET NEW_NODE -> DATA = VAL
Step 5: SET PTR = START
Step 6: Repeat Step 7 while PTR -> NEXT != START
Step 7:     PTR = PTR -> NEXT
    [END OF LOOP]
Step 8: SET NEW_NODE -> NEXT = START
Step 9: SET PTR -> NEXT = NEW_NODE
Step 10: SET START = NEW_NODE
Step 11: EXIT

```

Figure 6.30 Algorithm to insert a new node at the beginning

Figure 6.30 shows the algorithm to insert a new node at the beginning of a linked list. In Step 1, we first check whether memory is available for the new node. If the free memory has exhausted, then an **OVERFLOW** message is printed. Otherwise, if free memory cell is available, then we allocate space for the new node. Set its **DATA** part with the given **VAL** and the **NEXT** part is initialized with the address of the first node of the list, which is stored in **START**. Now, since the new node is added as the first node of the list, it will now be known as the **START** node, that is, the **START** pointer variable will now hold the address of the **NEW_NODE**.

While inserting a node in a circular linked list, we have to use a **while** loop to traverse to the last node of the list. Because the last node contains a pointer to **START**, its **NEXT** field is updated so that after insertion it points to the new node which will be now known as **START**.

Inserting a Node at the End of a Circular Linked List

Consider the linked list shown in Fig. 6.31. Suppose we want to add a new node with data 9 as the last node of the list. Then the following changes will be done in the linked list.

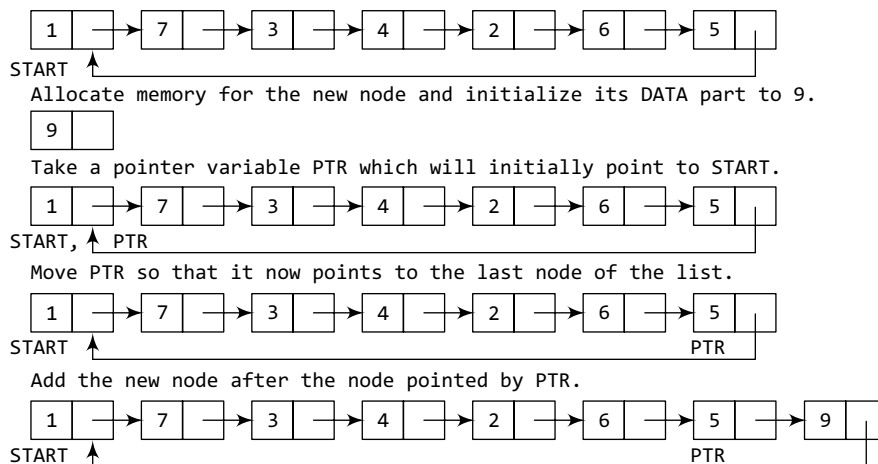


Figure 6.31 Inserting a new node at the end of a circular linked list

Figure 6.32 shows the algorithm to insert a new node at the end of a circular linked list. In Step 6, we take a pointer variable **PTR** and initialize it with **START**. That is, **PTR** now points to the first node of the linked list. In the **while** loop, we traverse through the linked list to reach the last node. Once we reach the last node, in Step 9, we change the **NEXT** pointer of the last node to store the address of the new node. Remember that the **NEXT** field of the new node contains the address of the first node which is denoted by **START**.

6.3.2 Deleting a Node from a Circular Linked List

In this section, we will discuss how a node is deleted from an already existing circular linked list. We will take two cases and then see how deletion is done in each case. Rest of the cases of

```

Step 1: IF AVAIL = NULL
        Write OVERFLOW
        Go to Step 10
    [END OF IF]
Step 2: SET NEW_NODE = AVAIL
Step 3: SET AVAIL = AVAIL -> NEXT
Step 4: SET NEW_NODE -> DATA = VAL
Step 5: SET NEW_NODE -> NEXT = START
Step 6: SET PTR = START
Step 7: Repeat Step 8 while PTR -> NEXT != START
Step 8:     SET PTR = PTR -> NEXT
    [END OF LOOP]
Step 9: SET PTR -> NEXT = NEW_NODE
Step 10: EXIT
    
```

Figure 6.32 Algorithm to insert a new node at the end

deletion are same as that given for singly linked lists.

Case 1: The first node is deleted.

Case 2: The last node is deleted.

Deleting the First Node from a Circular Linked List

Consider the circular linked list shown in Fig. 6.33. When we want to delete a node from the beginning of the list, then the following changes will be done in the linked list.

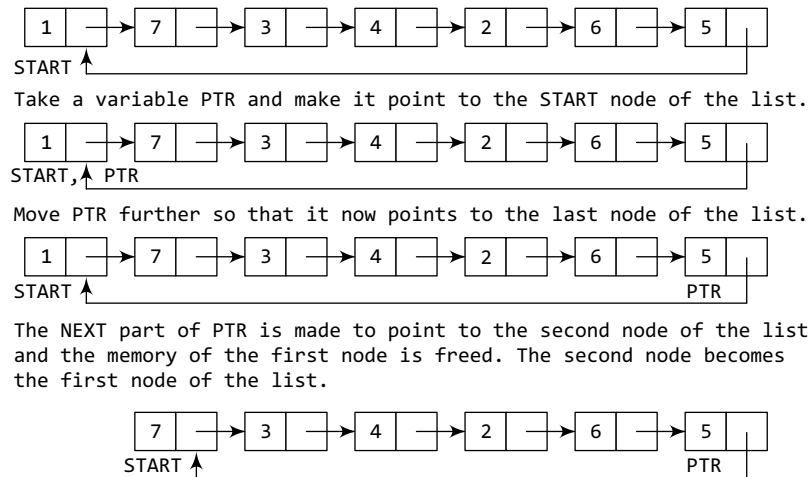


Figure 6.33 Deleting the first node from a circular linked list

Figure 6.34 shows the algorithm to delete the first node from a circular linked list. In Step 1 of the algorithm, we check if the linked list exists or not. If $START = NULL$, then it signifies that there are no nodes in the list and the control is transferred to the last statement of the algorithm.

However, if there are nodes in the linked list, then we use a pointer variable PTR which will be used to traverse the list to ultimately reach the last node. In Step 5, we change the next pointer

```

Step 1: IF START = NULL
        Write UNDERFLOW
        Go to Step 8
    [END OF IF]
Step 2: SET PTR = START
Step 3: Repeat Step 4 while PTR -> NEXT != START
Step 4:     SET PTR = PTR -> NEXT
    [END OF LOOP]
Step 5: SET PTR -> NEXT = START -> NEXT
Step 6: FREE START
Step 7: SET START = PTR -> NEXT
Step 8: EXIT
    
```

Figure 6.34 Algorithm to delete the first node

of the last node to point to the second node of the circular linked list. In Step 6, the memory occupied by the first node is freed. Finally, in Step 7, the second node now becomes the first node of the list and its address is stored in the pointer variable START.

Deleting the Last Node from a Circular Linked List

Consider the circular linked list shown in Fig. 6.35. Suppose we want to delete the last node from the linked list, then the following changes will be done in the linked list.

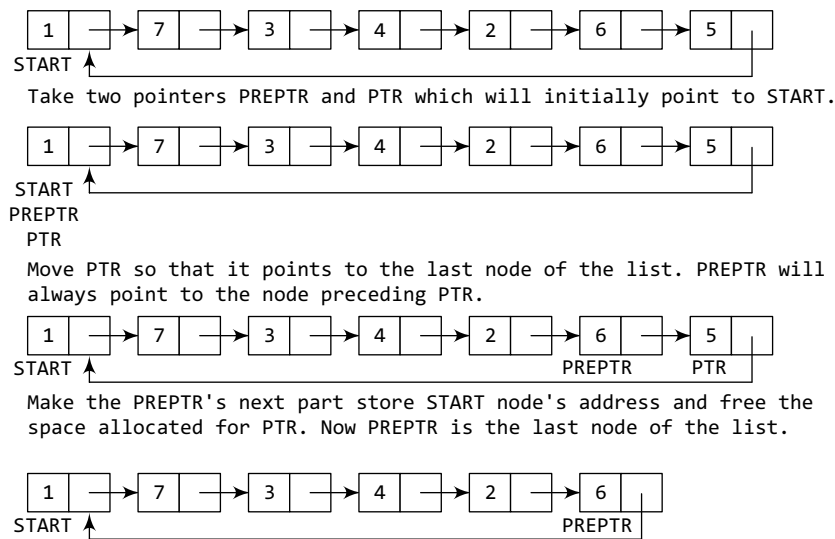


Figure 6.35 Deleting the last node from a circular linked list

```

Step 1: IF START = NULL
        Write UNDERFLOW
        Go to Step 8
    [END OF IF]
Step 2: SET PTR = START
Step 3: Repeat Steps 4 and 5 while PTR->NEXT != START
Step 4:     SET PREPTR = PTR
Step 5:     SET PTR = PTR->NEXT
    [END OF LOOP]
Step 6: SET PREPTR->NEXT = START
Step 7: FREE PTR
Step 8: EXIT

```

Figure 6.36 Algorithm to delete the last node

Figure 6.36 shows the algorithm to delete the last node from a circular linked list. In Step 2, we take a pointer variable PTR and initialize it with START. That is, PTR now points to the first node of the linked list. In the while loop, we take another pointer variable PREPTR such that PREPTR always points to one node before PTR. Once we reach the last node and the second last node, we set the next pointer of the second last node to START, so that it now becomes the (new) last node of the linked list. The memory of the previous last node is freed and returned to the free pool.

PROGRAMMING EXAMPLE

- Write a program to create a circular linked list. Perform insertion and deletion at the beginning and end of the list.

```

#include <stdio.h>
#include <conio.h>
#include <malloc.h>
struct node
{
    int data;
    struct node *next;
};
struct node *start = NULL;
struct node *create_cll(struct node *);
struct node *display(struct node *);
struct node *insert_beg(struct node *);
struct node *insert_end(struct node *);

```

```

struct node *delete_beg(struct node *);
struct node *delete_end(struct node *);
struct node *delete_after(struct node *);
struct node *delete_list(struct node *);
int main()
{
    int option;
    clrscr();
    do
    {
        printf("\n\n *****MAIN MENU *****");
        printf("\n 1: Create a list");
        printf("\n 2: Display the list");
        printf("\n 3: Add a node at the beginning");
        printf("\n 4: Add a node at the end");
        printf("\n 5: Delete a node from the beginning");
        printf("\n 6: Delete a node from the end");
        printf("\n 7: Delete a node after a given node");
        printf("\n 8: Delete the entire list");
        printf("\n 9: EXIT");
        printf("\n\n Enter your option : ");
        scanf("%d", &option);
        switch(option)
        {
            case 1: start = create_cll(start);
                    printf("\n CIRCULAR LINKED LIST CREATED");
                    break;
            case 2: start = display(start);
                    break;
            case 3: start = insert_beg(start);
                    break;
            case 4: start = insert_end(start);
                    break;
            case 5: start = delete_beg(start);
                    break;
            case 6: start = delete_end(start);
                    break;
            case 7: start = delete_after(start);
                    break;
            case 8: start = delete_list(start);
                    printf("\n CIRCULAR LINKED LIST DELETED");
                    break;
        }
    }while(option !=9);
    getch();
    return 0;
}

struct node *create_cll(struct node *start)
{
    struct node *new_node, *ptr;
    int num;
    printf("\n Enter -1 to end");
    printf("\n Enter the data : ");
    scanf("%d", &num);
    while(num!=-1)
    {
        new_node = (struct node*)malloc(sizeof(struct node));
        new_node->data = num;
        if(start == NULL)
        {
            new_node->next = new_node;

```



```
        start = new_node;
    }
    else
    {
        ptr = start;
        while(ptr->next != start)
            ptr = ptr->next;
        ptr->next = new_node;
        new_node->next = start;
    }
    printf("\n Enter the data : ");
    scanf("%d", &num);
}
return start;
}
struct node *display(struct node *start)
{
    struct node *ptr;
    ptr=start;
    while(ptr->next != start)
    {
        printf("\t %d", ptr->data);
        ptr = ptr->next;
    }
    printf("\t %d", ptr->data);
    return start;
}
struct node *insert_beg(struct node *start)
{
    struct node *new_node, *ptr;
    int num;
    printf("\n Enter the data : ");
    scanf("%d", &num);
    new_node = (struct node *)malloc(sizeof(struct node));
    new_node->data = num;
    ptr = start;
    while(ptr->next != start)
        ptr = ptr->next;
    ptr->next = new_node;
    new_node->next = start;
    start = new_node;
    return start;
}
struct node *insert_end(struct node *start)
{
    struct node *ptr, *new_node;
    int num;
    printf("\n Enter the data : ");
    scanf("%d", &num);
    new_node = (struct node *)malloc(sizeof(struct node));
    new_node->data = num;
    ptr = start;
    while(ptr->next != start)
        ptr = ptr->next;
    ptr->next = new_node;
    new_node->next = start;
    return start;
}
struct node *delete_beg(struct node *start)
{
    struct node *ptr;
    ptr = start;
```

```

        while(ptr->next != start)
            ptr = ptr->next;
        ptr->next = start->next;
        free(start);
        start = ptr->next;
        return start;
    }
    struct node *delete_end(struct node *start)
    {
        struct node *ptr, *preptr;
        ptr = start;
        while(ptr->next != start)
        {
            preptr = ptr;
            ptr = ptr->next;
        }
        preptr->next = ptr->next;
        free(ptr);
        return start;
    }
    struct node *delete_after(struct node *start)
    {
        struct node *ptr, *preptr;
        int val;
        printf("\n Enter the value after which the node has to deleted : ");
        scanf("%d", &val);
        ptr = start;
        preptr = ptr;
        while(preptr->data != val)
        {
            preptr = ptr;
            ptr = ptr->next;
        }
        preptr->next = ptr->next;
        if(ptr == start)
            start = preptr->next;
        free(ptr);
        return start;
    }
    struct node *delete_list(struct node *start)
    {
        struct node *ptr;
        ptr = start;
        while(ptr->next != start)
            start = delete_end(start);
        free(start);
        return start;
    }
}

```

Output

```

*****MAIN MENU *****
1: Create a list
2: Display the list
3: Add a node at the beginning
-----
8: Delete the entire list
9: EXIT
Enter your option : 1
Enter -1 to end
Enter the data: 1
Enter the data: 2

```

```

Enter the data: 4
Enter the data: -1
CIRCULAR LINKED LIST CREATED
Enter your option : 3
Enter your option : 5
Enter your option : 2
5 1 2 4
Enter your option : 9

```

6.4 DOUBLY LINKED LISTS

A doubly linked list or a two-way linked list is a more complex type of linked list which contains a pointer to the next as well as the previous node in the sequence. Therefore, it consists of three parts—data, a pointer to the next node, and a pointer to the previous node as shown in Fig. 6.37.

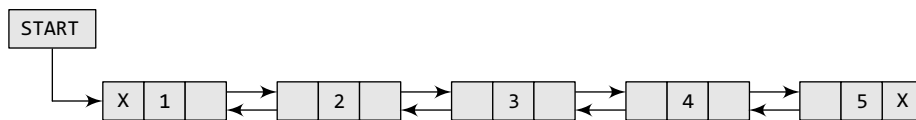


Figure 6.37 Doubly linked list

In C, the structure of a doubly linked list can be given as,

```

struct node
{
    struct node *prev;
    int data;
    struct node *next;
};

```

The PREV field of the first node and the NEXT field of the last node will contain NULL. The PREV field is used to store the address of the preceding node, which enables us to traverse the list in the backward direction.

Thus, we see that a doubly linked list calls for more space per node and more expensive basic operations. However, a doubly linked list provides the ease to manipulate the elements of the list as it maintains pointers to nodes in both the directions (forward and backward). The main advantage of using a doubly linked list is that it makes searching twice as efficient. Let us view how a doubly linked list is maintained in the memory. Consider Fig. 6.38.

In the figure, we see that a variable START is used to store the address of the first node. In this example, START = 1, so the first data is stored at address 1, which is H. Since this is the first node, it has no previous node and hence stores NULL or -1 in the PREV field. We will traverse the list until we reach a position where the NEXT entry contains -1 or NULL. This denotes the end of the linked list. When we traverse the DATA and NEXT in this manner, we will finally see that the linked list in the above example stores characters that when put together form the word HELLO.

START	1			
	1	H	-1	3
	2			
	3	E	1	6
	4			
	5			
	6	L	3	7
	7	L	6	9
	8			
	9	O	7	-1

Figure 6.38 Memory representation of a doubly linked list

6.4.1 Inserting a New Node in a Doubly Linked List

In this section, we will discuss how a new node is added into an already existing doubly linked list. We will take four cases and then see how insertion is done in each case.

Case 1: The new node is inserted at the beginning.

- Case 2: The new node is inserted at the end.
 Case 3: The new node is inserted after a given node.
 Case 4: The new node is inserted before a given node.

Inserting a Node at the Beginning of a Doubly Linked List

Consider the doubly linked list shown in Fig. 6.39. Suppose we want to add a new node with data 9 as the first node of the list. Then the following changes will be done in the linked list.

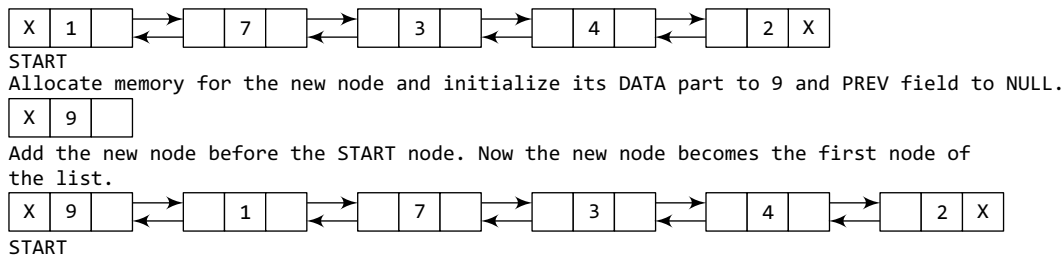


Figure 6.39 Inserting a new node at the beginning of a doubly linked list

```

Step 1: IF AVAIL = NULL
        Write OVERFLOW
        Go to Step 9
    [END OF IF]
Step 2: SET NEW_NODE = AVAIL
Step 3: SET AVAIL = AVAIL -> NEXT
Step 4: SET NEW_NODE -> DATA = VAL
Step 5: SET NEW_NODE -> PREV = NULL
Step 6: SET NEW_NODE -> NEXT = START
Step 7: SET START -> PREV = NEW_NODE
Step 8: SET START = NEW_NODE
Step 9: EXIT
    
```

Figure 6.40 Algorithm to insert a new node at the beginning

Figure 6.40 shows the algorithm to insert a new node at the beginning of a doubly linked list. In Step 1, we first check whether memory is available for the new node. If the free memory has exhausted, then an OVERFLOW message is printed. Otherwise, if free memory cell is available, then we allocate space for the new node. Set its DATA part with the given VAL and the NEXT part is initialized with the address of the first node of the list, which is stored in START. Now, since the new node is added as the first node of the list, it will now be known as the START node, that is, the START pointer variable will now hold the address of NEW_NODE.

Inserting a Node at the End of a Doubly Linked List

Consider the doubly linked list shown in Fig. 6.41. Suppose we want to add a new node with data 9 as the last node of the list. Then the following changes will be done in the linked list.

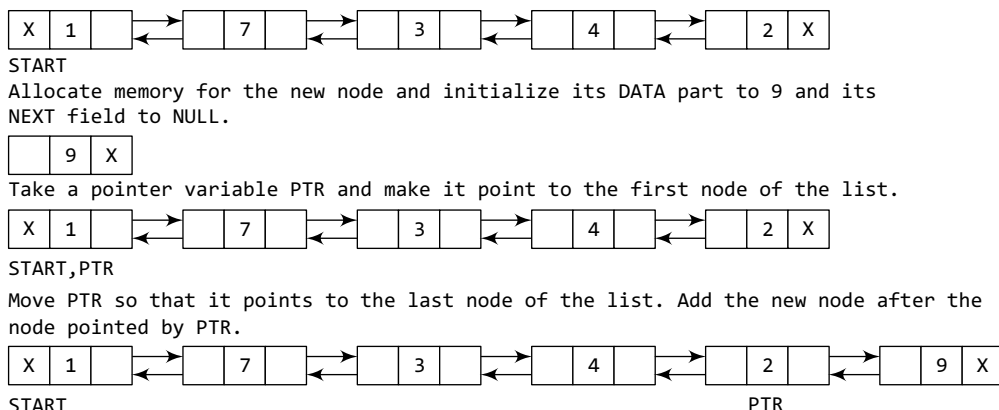


Figure 6.41 Inserting a new node at the end of a doubly linked list

Figure 6.42 shows the algorithm to insert a new node at the end of a doubly linked list. In Step 6, we take a pointer variable PTR and initialize it with START. In the while loop, we traverse through the linked list to reach the last node. Once we reach the last node, in Step 9, we change the NEXT pointer of the last node to store the address of the new node. Remember that the NEXT field of the new node contains NULL which signifies the end of the linked list. The PREV field of the NEW_NODE will be set so that it points to the node pointed by PTR (now the second last node of the list).

```

Step 1: IF AVAIL = NULL
        Write OVERFLOW
        Go to Step 11
    [END OF IF]
Step 2: SET NEW_NODE = AVAIL
Step 3: SET AVAIL = AVAIL -> NEXT
Step 4: SET NEW_NODE -> DATA = VAL
Step 5: SET NEW_NODE -> NEXT = NULL
Step 6: SET PTR = START
Step 7: Repeat Step 8 while PTR -> NEXT != NULL
Step 8:     SET PTR = PTR -> NEXT
    [END OF LOOP]
Step 9: SET PTR -> NEXT = NEW_NODE
Step 10: SET NEW_NODE -> PREV = PTR
Step 11: EXIT

```

Figure 6.42 Algorithm to insert a new node at the end

```

Step 1: IF AVAIL = NULL
        Write OVERFLOW
        Go to Step 12
    [END OF IF]
Step 2: SET NEW_NODE = AVAIL
Step 3: SET AVAIL = AVAIL -> NEXT
Step 4: SET NEW_NODE -> DATA = VAL
Step 5: SET PTR = START
Step 6: Repeat Step 7 while PTR -> DATA != NUM
Step 7:     SET PTR = PTR -> NEXT
    [END OF LOOP]
Step 8: SET NEW_NODE -> NEXT = PTR -> NEXT
Step 9: SET NEW_NODE -> PREV = PTR
Step 10: SET PTR -> NEXT = NEW_NODE
Step 11: SET PTR -> NEXT -> PREV = NEW_NODE
Step 12: EXIT

```

Figure 6.43 Algorithm to insert a new node after a given node

Inserting a Node After a Given Node in a Doubly Linked List

Consider the doubly linked list shown in Fig. 6.44. Suppose we want to add a new node with value 9 after the node containing 3. Before discussing the changes that will be done in the linked list, let us first look at the algorithm shown in Fig. 6.43.

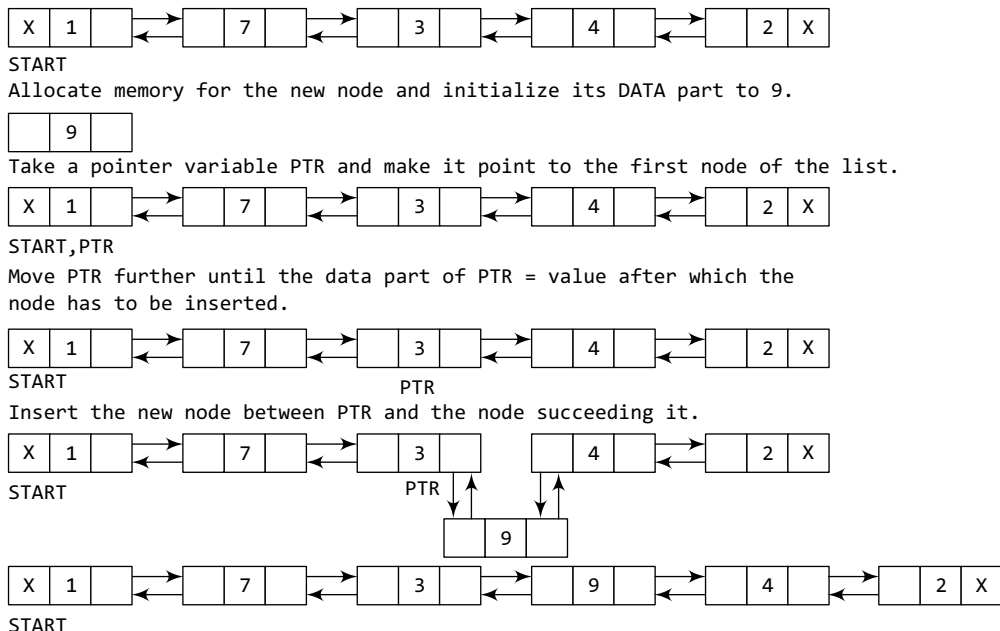


Figure 6.44 Inserting a new node after a given node in a doubly linked list

```

Step 1: IF AVAIL = NULL
        Write OVERFLOW
        Go to Step 12
    [END OF IF]
Step 2: SET NEW_NODE = AVAIL
Step 3: SET AVAIL = AVAIL -> NEXT
Step 4: SET NEW_NODE->DATA = VAL
Step 5: SET PTR = START
Step 6: Repeat Step 7 while PTR->DATA != NUM
Step 7:     SET PTR = PTR->NEXT
    [END OF LOOP]
Step 8: SET NEW_NODE->NEXT = PTR
Step 9: SET NEW_NODE->PREV = PTR->PREV
Step 10: SET PTR->PREV = NEW_NODE
Step 11: SET PTR -> PREV -> NEXT = NEW_NODE
Step 12: EXIT

```

Figure 6.45 Algorithm to insert a new node before a given node

changes that will be done in the linked list, let us first look at the algorithm shown in Fig. 6.45.

In Step 1, we first check whether memory is available for the new node. In Step 5, we take a pointer variable `PTR` and initialize it with `START`. That is, `PTR` now points to the first node of the linked list. In the `while` loop, we traverse through the linked list to reach the node that has its value equal to `NUM`. We need to reach this node because the new node will be inserted before this node. Once we reach this node, we change the `NEXT` and `PREV` fields in such a way that the new node is inserted before the desired node.

Figure 6.43 shows the algorithm to insert a new node after a given node in a doubly linked list. In Step 5, we take a pointer PTR and initialize it with START. That is, PTR now points to the first node of the linked list. In the while loop, we traverse through the linked list to reach the node that has its value equal to NUM. We need to reach this node because the new node will be inserted after this node. Once we reach this node, we change the NEXT and PREV fields in such a way that the new node is inserted after the desired node.

Inserting a Node Before a Given Node in a Doubly Linked List

Consider the doubly linked list shown in Fig. 6.46. Suppose we want to add a new node with value 9 before the node containing 3. Before discussing the

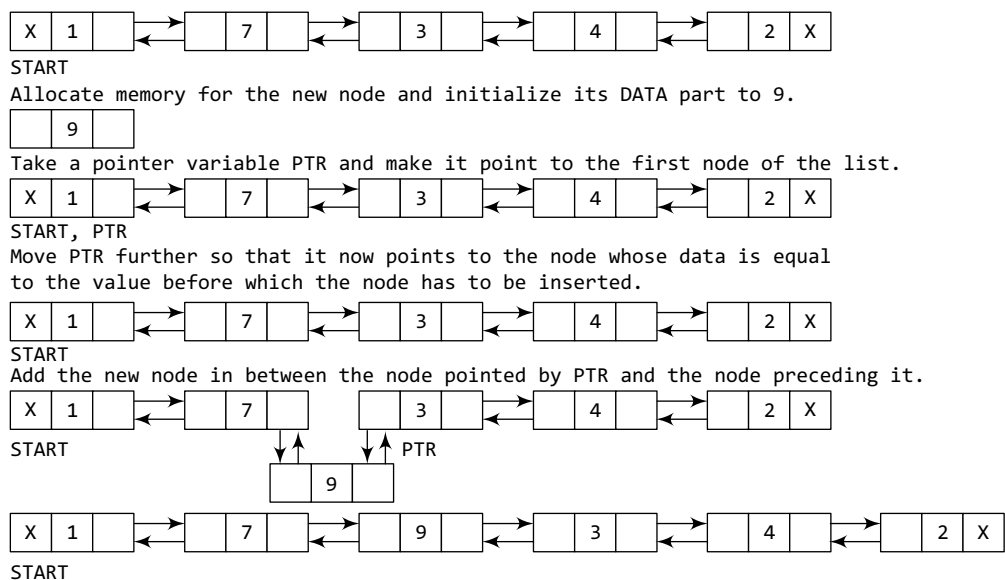


Figure 6.46 Inserting a new node before a given node in a doubly linked list

6.4.2 Deleting a Node from a Doubly Linked List

In this section, we will see how a node is deleted from an already existing doubly linked list. We will take four cases and then see how deletion is done in each case.

- Case 1: The first node is deleted.
- Case 2: The last node is deleted.
- Case 3: The node after a given node is deleted.
- Case 4: The node before a given node is deleted.

Deleting the First Node from a Doubly Linked List

Consider the doubly linked list shown in Fig. 6.47. When we want to delete a node from the beginning of the list, then the following changes will be done in the linked list.

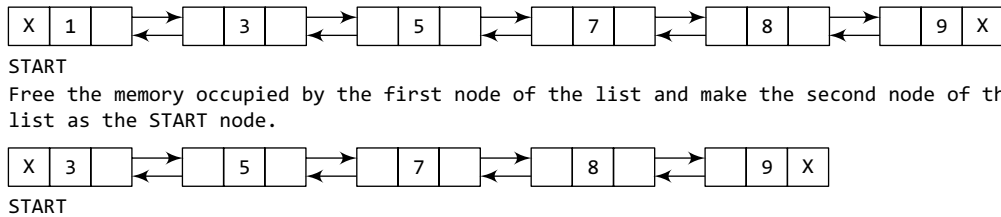


Figure 6.47 Deleting the first node from a doubly linked list

Figure 6.48 shows the algorithm to delete the first node of a doubly linked list. In Step 1 of the algorithm, we check if the linked list exists or not. If $START = NULL$, then it signifies that there are no nodes in the list and the control is transferred to the last statement of the algorithm.

```

Step 1: IF  $START = NULL$ 
        Write UNDERFLOW
        Go to Step 6
    [END OF IF]
Step 2: SET  $PTR = START$ 
Step 3: SET  $START = START \rightarrow NEXT$ 
Step 4: SET  $START \rightarrow PREV = NULL$ 
Step 5: FREE  $PTR$ 
Step 6: EXIT
  
```

However, if there are nodes in the linked list, then we use a temporary pointer variable PTR that is set to point to the first node of the list. For this, we initialize PTR with $START$ that stores the address of the first node of the list. In Step 3, $START$ is made to point to the next node in sequence and finally the memory occupied by PTR (initially the first node of the list) is freed and returned to the free pool.

Figure 6.48 Algorithm to delete the first node

Deleting the Last Node from a Doubly Linked List

Consider the doubly linked list shown in Fig. 6.49. Suppose we want to delete the last node from the linked list, then the following changes will be done in the linked list.

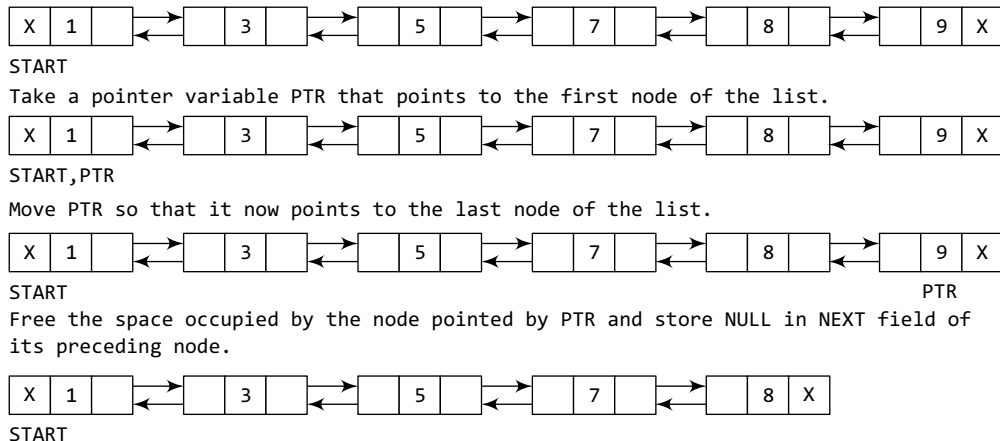


Figure 6.49 Deleting the last node from a doubly linked list

```

Step 1: IF START = NULL
        Write UNDERFLOW
        Go to Step 7
    [END OF IF]
Step 2: SET PTR = START
Step 3: Repeat Step 4 while PTR->NEXT != NULL
Step 4:     SET PTR = PTR->NEXT
    [END OF LOOP]
Step 5: SET PTR->PREV->NEXT = NULL
Step 6: FREE PTR
Step 7: EXIT

```

Figure 6.50 Algorithm to delete the last node

Figure 6.50 shows the algorithm to delete the last node of a doubly linked list. In Step 2, we take a pointer variable PTR and initialize it with START. That is, PTR now points to the first node of the linked list. The while loop traverses through the list to reach the last node. Once we reach the last node, we can also access the second last node by taking its address from the PREV field of the last node. To delete the last node, we simply have to set the next field of second last node to NULL, so that it now becomes the (new) last node of the linked list. The memory of the previous last node is freed and returned to the free pool.

Deleting the Node After a Given Node in a Doubly Linked List

Consider the doubly linked list shown in Fig. 6.51. Suppose we want to delete the node that succeeds the node which contains data value 4. Then the following changes will be done in the linked list.

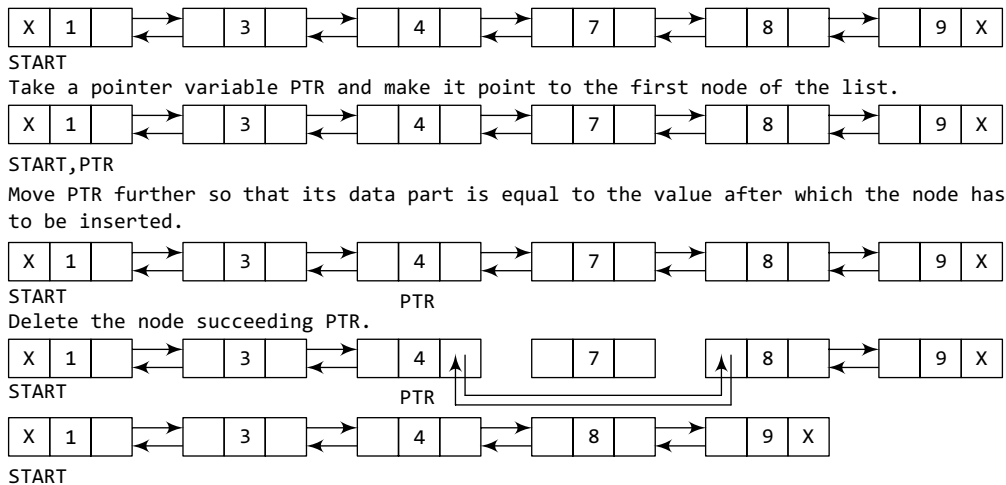


Figure 6.51 Deleting the node after a given node in a doubly linked list

```

Step 1: IF START = NULL
        Write UNDERFLOW
        Go to Step 9
    [END OF IF]
Step 2: SET PTR = START
Step 3: Repeat Step 4 while PTR->DATA != NUM
Step 4:     SET PTR = PTR->NEXT
    [END OF LOOP]
Step 5: SET TEMP = PTR->NEXT
Step 6: SET PTR->NEXT = TEMP->NEXT
Step 7: SET TEMP->NEXT->PREV = PTR
Step 8: FREE TEMP
Step 9: EXIT

```

Figure 6.52 Algorithm to delete a node after a given node

Figure 6.52 shows the algorithm to delete a node after a given node of a doubly linked list. In Step 2, we take a pointer variable PTR and initialize it with START. That is, PTR now points to the first node of the doubly linked list. The while loop traverses through the linked list to reach the given node. Once we reach the node containing VAL, the node succeeding it can be easily accessed by using the address stored in its NEXT field. The NEXT field of the given node is set to contain the contents in the NEXT field of the succeeding node. Finally, the memory of the node succeeding the given node is freed and returned to the free pool.

Deleting the Node Before a Given Node in a Doubly Linked List

Consider the doubly linked list shown in Fig. 6.53. Suppose we want to delete the node preceding the node with value 4. Before discussing the changes that will be done in the linked list, let us first look at the algorithm.

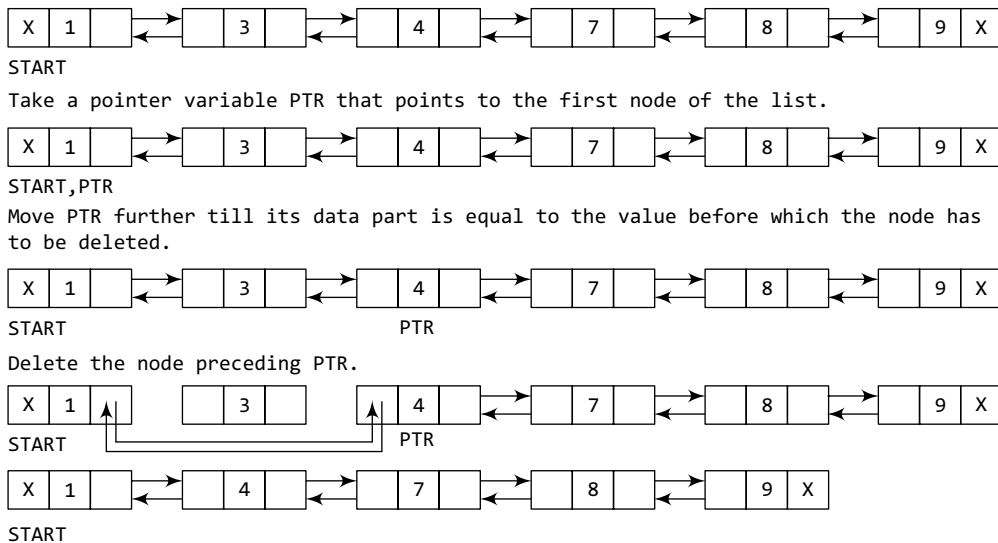


Figure 6.53 Deleting a node before a given node in a doubly linked list

```

Step 1: IF START = NULL
        Write UNDERFLOW
        Go to Step 9
    [END OF IF]
Step 2: SET PTR = START
Step 3: Repeat Step 4 while PTR->DATA != NUM
Step 4:     SET PTR = PTR->NEXT
    [END OF LOOP]
Step 5: SET TEMP = PTR->PREV
Step 6: SET TEMP->PREV->NEXT = PTR
Step 7: SET PTR->PREV = TEMP->PREV
Step 8: FREE TEMP
Step 9: EXIT

```

Figure 6.54 Algorithm to delete a node before a given node

Figure 6.54 shows the algorithm to delete a node before a given node of a doubly linked list. In Step 2, we take a pointer variable PTR and initialize it with START. That is, PTR now points to the first node of the linked list. The while loop traverses through the linked list to reach the desired node. Once we reach the node containing VAL, the PREV field of PTR is set to contain the address of the node preceding the node which comes before PTR. The memory of the node preceding PTR is freed and returned to the free pool.

Hence, we see that we can insert or delete a node in a constant number of operations given only that node's address. Note that this is not possible in the

case of a singly linked list which requires the previous node's address also to perform the same operation.

PROGRAMMING EXAMPLE

- Write a program to create a doubly linked list and perform insertions and deletions in all cases.

```

#include <stdio.h>
#include <conio.h>
#include <malloc.h>

```

```

struct node
{
    struct node *next;
    int data;
    struct node *prev;
};
struct node *start = NULL;
struct node *create_ll(struct node *);
struct node *display(struct node *);
struct node *insert_beg(struct node *);
struct node *insert_end(struct node *);
struct node *insert_before(struct node *);
struct node *insert_after(struct node *);
struct node *delete_beg(struct node *);
struct node *delete_end(struct node *);
struct node *delete_before(struct node *);
struct node *delete_after(struct node *);
struct node *delete_list(struct node *);
int main()
{
    int option;
    clrscr();
    do
    {
        printf("\n\n *****MAIN MENU *****");
        printf("\n 1: Create a list");
        printf("\n 2: Display the list");
        printf("\n 3: Add a node at the beginning");
        printf("\n 4: Add a node at the end");
        printf("\n 5: Add a node before a given node");
        printf("\n 6: Add a node after a given node");
        printf("\n 7: Delete a node from the beginning");
        printf("\n 8: Delete a node from the end");
        printf("\n 9: Delete a node before a given node");
        printf("\n 10: Delete a node after a given node");
        printf("\n 11: Delete the entire list");
        printf("\n 12: EXIT");
        printf("\n\n Enter your option : ");
        scanf("%d", &option);
        switch(option)
        {
            case 1: start = create_ll(start);
                    printf("\n DOUBLY LINKED LIST CREATED");
                    break;
            case 2: start = display(start);
                    break;
            case 3: start = insert_beg(start);
                    break;
            case 4: start = insert_end(start);
                    break;
            case 5: start = insert_before(start);
                    break;
            case 6: start = insert_after(start);
                    break;
            case 7: start = delete_beg(start);
                    break;
            case 8: start = delete_end(start);
                    break;
            case 9: start = delete_before(start);
                    break;
            case 10: start = delete_after(start);

```

```

                                break;
                                case 11: start = delete_list(start);
                                printf("\n DOUBLY LINKED LIST DELETED");
                                break;
                                }
                                }while(option != 12);
                                getch();
                                return 0;
                                }
                                struct node *create_ll(struct node *start)
                                {
                                        struct node *new_node, *ptr;
                                        int num;
                                        printf("\n Enter -1 to end");
                                        printf("\n Enter the data : ");
                                        scanf("%d", &num);
                                        while(num != -1)
                                        {
                                                if(start == NULL)
                                                {
                                                        new_node = (struct node*)malloc(sizeof(struct node));
                                                        new_node->prev = NULL;
                                                        new_node->data = num;
                                                        new_node->next = NULL;
                                                        start = new_node;
                                                }
                                                else
                                                {
                                                        ptr=start;
                                                        new_node = (struct node*)malloc(sizeof(struct node));
                                                        new_node->data=num;
                                                        while(ptr->next!=NULL)
                                                                ptr = ptr->next;
                                                        ptr->next = new_node;
                                                        new_node->prev=ptr;
                                                        new_node->next=NULL;
                                                }
                                                printf("\n Enter the data : ");
                                                scanf("%d", &num);
                                        }
                                        return start;
                                }
                                struct node *display(struct node *start)
                                {
                                        struct node *ptr;
                                        ptr=start;
                                        while(ptr!=NULL)
                                        {
                                                printf("\t %d", ptr->data);
                                                ptr = ptr->next;
                                        }
                                        return start;
                                }
                                struct node *insert_beg(struct node *start)
                                {
                                        struct node *new_node;
                                        int num;
                                        printf("\n Enter the data : ");
                                        scanf("%d", &num);
                                        new_node = (struct node *)malloc(sizeof(struct node));
                                        new_node->data = num;

```

```

        start->prev = new_node;
        new_node->next = start;
        new_node->prev = NULL;
        start = new_node;
        return start;
    }
    struct node *insert_end(struct node *start)
    {
        struct node *ptr, *new_node;
        int num;
        printf("\n Enter the data : ");
        scanf("%d", &num);
        new_node = (struct node *)malloc(sizeof(struct node));
        new_node->data = num;
        ptr=start;
        while(ptr->next != NULL)
            ptr = ptr->next;
        ptr->next = new_node;
        new_node->prev = ptr;
        new_node->next = NULL;
        return start;
    }
    struct node *insert_before(struct node *start)
    {
        struct node *new_node, *ptr;
        int num, val;
        printf("\n Enter the data : ");
        scanf("%d", &num);
        printf("\n Enter the value before which the data has to be inserted:");
        scanf("%d", &val);
        new_node = (struct node *)malloc(sizeof(struct node));
        new_node->data = num;
        ptr = start;
        while(ptr->data != val)
            ptr = ptr->next;
        new_node->next = ptr;
        new_node->prev = ptr->prev;
        ptr->prev->next = new_node;
        ptr->prev = new_node;
        return start;
    }
    struct node *insert_after(struct node *start)
    {
        struct node *new_node, *ptr;
        int num, val;
        printf("\n Enter the data : ");
        scanf("%d", &num);
        printf("\n Enter the value after which the data has to be inserted:");
        scanf("%d", &val);
        new_node = (struct node *)malloc(sizeof(struct node));
        new_node->data = num;
        ptr = start;
        while(ptr->data != val)
            ptr = ptr->next;
        new_node->prev = ptr;
        new_node->next = ptr->next;
        ptr->next->prev = new_node;
        ptr->next = new_node;
        return start;
    }
}

```

```
struct node *delete_beg(struct node *start)
{
    struct node *ptr;
    ptr = start;
    start = start->next;
    start->prev = NULL;
    free(ptr);
    return start;
}
struct node *delete_end(struct node *start)
{
    struct node *ptr;
    ptr = start;
    while(ptr->next != NULL)
        ptr = ptr->next;
    ptr->prev->next = NULL;
    free(ptr);
    return start;
}
struct node *delete_after(struct node *start)
{
    struct node *ptr, *temp;
    int val;
    printf("\n Enter the value after which the node has to deleted : ");
    scanf("%d", &val);
    ptr = start;
    while(ptr->data != val)
        ptr = ptr->next;
    temp = ptr->next;
    ptr->next = temp->next;
    temp->next->prev = ptr;
    free(temp);
    return start;
}
struct node *delete_before(struct node *start)
{
    struct node *ptr, *temp;
    int val;
    printf("\n Enter the value before which the node has to deleted:");
    scanf("%d", &val);
    ptr = start;
    while(ptr->data != val)
        ptr = ptr->next;
    temp = ptr->prev;
    if(temp == start)
        start = delete_beg(start);
    else
    {
        ptr->prev = temp->prev;
        temp->prev->next = ptr;
    }
    free(temp);
    return start;
}
struct node *delete_list(struct node *start)
{
    while(start != NULL)
        start = delete_beg(start);
    return start;
}
```

Output

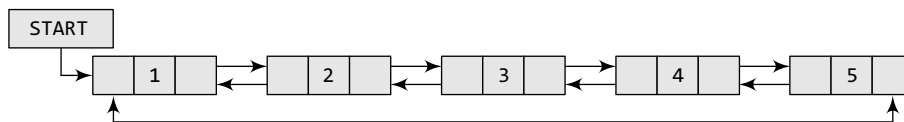
```

*****MAIN MENU *****
1: Create a list
2: Display the list
-----
11: Delete the entire list
12: EXIT
Enter your option : 1
Enter -1 to end
Enter the data: 1
Enter the data: 3
Enter the data: 4
Enter the data: -1
DOUBLY LINKED LIST CREATED
Enter your option : 12

```

6.5 CIRCULAR DOUBLY LINKED LISTS

A circular doubly linked list or a circular two-way linked list is a more complex type of linked list which contains a pointer to the next as well as the previous node in the sequence. The difference between a doubly linked and a circular doubly linked list is same as that exists between a singly linked list and a circular linked list. The circular doubly linked list does not contain NULL in the previous field of the first node and the next field of the last node. Rather, the next field of the last node stores the address of the first node of the list, i.e., START. Similarly, the previous field of the first field stores the address of the last node. A circular doubly linked list is shown in Fig. 6.55.

**Figure 6.55** Circular doubly linked list

Since a circular doubly linked list contains three parts in its structure, it calls for more space per node and more expensive basic operations. However, a circular doubly linked list provides the ease to manipulate the elements of the list as it maintains pointers to nodes in both the directions (forward and backward). The main advantage of using a circular doubly linked list is that it makes search operation twice as efficient.

START
1

	DATA	PREV	Next
1	H	9	3
2			
3	E	1	6
4			
5			
6	L	3	7
7	L	6	9
8			
9	O	7	1

Figure 6.56 Memory representation of a circular doubly linked list

Let us view how a circular doubly linked list is maintained in the memory. Consider Fig. 6.56. In the figure, we see that a variable START is used to store the address of the first node. Here in this example, START = 1, so the first data is stored at address 1, which is H. Since this is the first node, it stores the address of the last node of the list in its previous field. The corresponding NEXT stores the address of the next node, which is 3. So, we will look at address 3 to fetch the next data item. The previous field will contain the address of the first node. The second data element obtained from address 3 is E. We repeat this procedure until we reach a position where the NEXT entry stores the address of the first element of the list. This denotes the end of the linked list, that is, the node that contains the address of the first node is actually the last node of the list.

6.5.1 Inserting a New Node in a Circular Doubly Linked List

In this section, we will see how a new node is added into an already existing circular doubly linked list. We will take two cases and then see how insertion is done in each case. Rest of the cases are similar to that given for doubly linked lists.

Case 1: The new node is inserted at the beginning.

Case 2: The new node is inserted at the end.

Inserting a Node at the Beginning of a Circular Doubly Linked List

Consider the circular doubly linked list shown in Fig. 6.57. Suppose we want to add a new node with data 9 as the first node of the list. Then, the following changes will be done in the linked list.

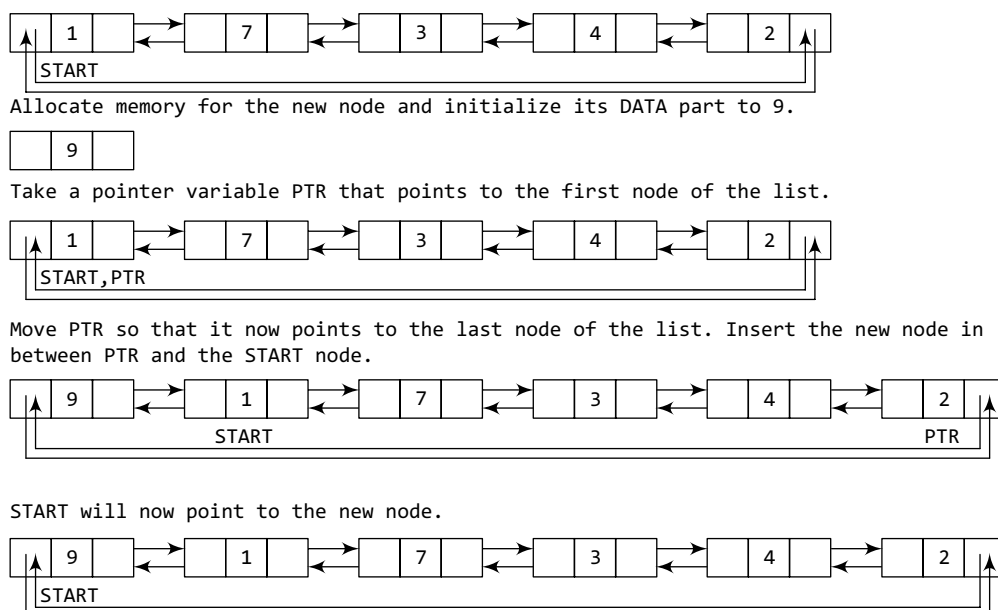


Figure 6.57 Inserting a new node at the beginning of a circular doubly linked list

```

Step 1: IF AVAIL = NULL
        Write OVERFLOW
        Go to Step 13
    [END OF IF]
Step 2: SET NEW_NODE = AVAIL
Step 3: SET AVAIL = AVAIL->NEXT
Step 4: SET NEW_NODE->DATA = VAL
Step 5: SET PTR = START
Step 6: Repeat Step 7 while PTR->NEXT != START
Step 7:     SET PTR = PTR->NEXT
    [END OF LOOP]
Step 8: SET PTR->NEXT = NEW_NODE
Step 9: SET NEW_NODE->PREV = PTR
Step 10: SET NEW_NODE->NEXT = START
Step 11: SET START->PREV = NEW_NODE
Step 12: SET START = NEW_NODE
Step 13: EXIT

```

Figure 6.58 Algorithm to insert a new node at the beginning

Figure 6.58 shows the algorithm to insert a new node at the beginning of a circular doubly linked list. In Step 1, we first check whether memory is available for the new node. If the free memory has exhausted, then an OVERFLOW message is printed. Otherwise, we allocate space for the new node. Set its data part with the given VAL and its next part is initialized with the address of the first node of the list, which is stored in START. Now since the new node is added as the first node of the list, it will now be known as the START node, that is, the START pointer variable will now hold the address of NEW_NODE. Since it is a circular doubly linked list, the PREV field of the NEW_NODE is set to contain the address of the last node.

Inserting a Node at the End of a Circular Doubly Linked List

Consider the circular doubly linked list shown in Fig. 6.59. Suppose we want to add a new node with data 9 as the last node of the list. Then the following changes will be done in the linked list.

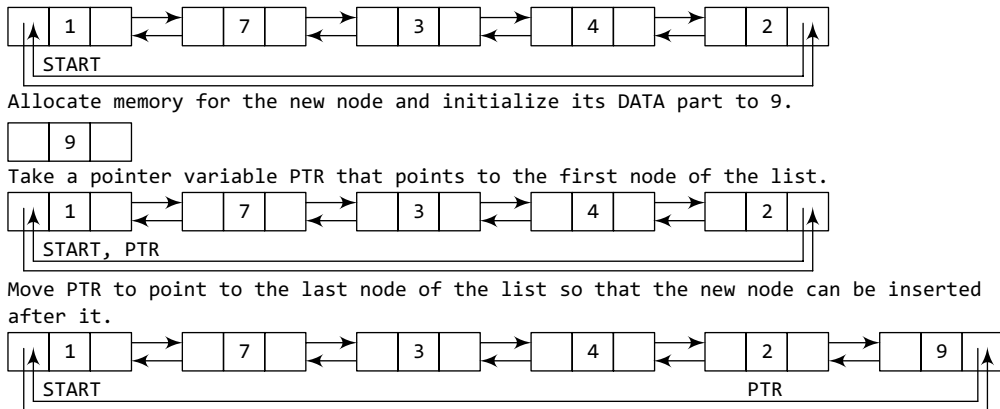


Figure 6.59 Inserting a new node at the end of a circular doubly linked list

Figure 6.60 shows the algorithm to insert a new node at the end of a circular doubly linked list. In Step 6, we take a pointer variable PTR and initialize it with START. That is, PTR now points to the first node of the linked list. In the while loop, we traverse through the linked list to reach the last node. Once we reach the last node, in Step 9, we change the NEXT pointer of the last node to store the address of the new node. The PREV field of the NEW_NODE will be set so that it points to the node pointed by PTR (now the second last node of the list).

6.5.2 Deleting a Node from a Circular Doubly Linked List

In this section, we will see how a node is deleted from an already existing circular doubly linked list. We will take two cases and then see how deletion is done in each case. Rest of the cases are same as that given for doubly linked lists.

```

Step 1: IF AVAIL = NULL
        Write OVERFLOW
        Go to Step 12
    [END OF IF]
Step 2: SET NEW_NODE = AVAIL
Step 3: SET AVAIL = AVAIL -> NEXT
Step 4: SET NEW_NODE -> DATA = VAL
Step 5: SET NEW_NODE -> NEXT = START
Step 6: SET PTR = START
Step 7: Repeat Step 8 while PTR -> NEXT != START
Step 8:     SET PTR = PTR -> NEXT
    [END OF LOOP]
Step 9: SET PTR -> NEXT = NEW_NODE
Step 10: SET NEW_NODE -> PREV = PTR
Step 11: SET START -> PREV = NEW_NODE
Step 12: EXIT

```

Figure 6.60 Algorithm to insert a new node at the end

Case 1: The first node is deleted.

Case 2: The last node is deleted.

Deleting the First Node from a Circular Doubly Linked List

Consider the circular doubly linked list shown in Fig. 6.61. When we want to delete a node from the beginning of the list, then the following changes will be done in the linked list.

Figure 6.62 shows the algorithm to delete the first node from a circular doubly linked list. In Step 1 of the algorithm, we check if the linked list exists or not. If START = NULL, then it signifies that there are no nodes in the list and the control is transferred to the last statement of the algorithm.

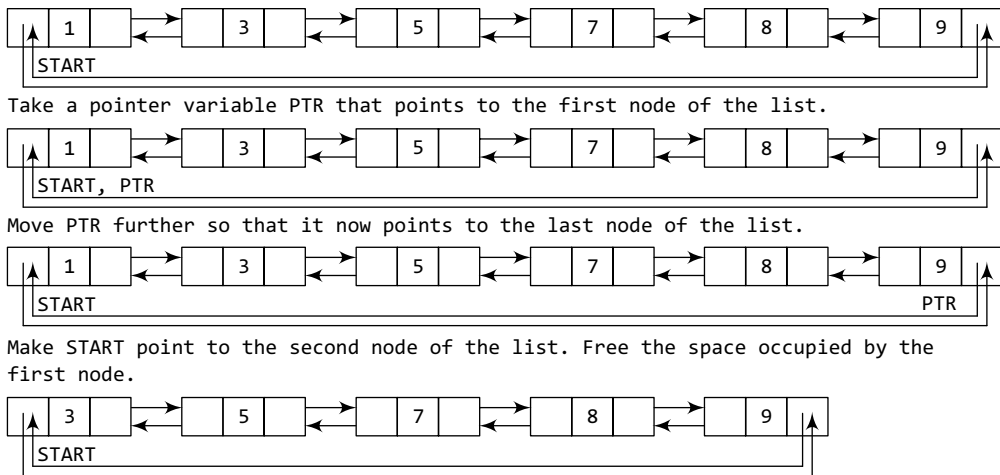


Figure 6.61 Deleting the first node from a circular doubly linked list

```

Step 1: IF START = NULL
        Write UNDERFLOW
        Go to Step 8
    [END OF IF]
Step 2: SET PTR = START
Step 3: Repeat Step 4 while PTR->NEXT != START
Step 4:     SET PTR = PTR->NEXT
    [END OF LOOP]
Step 5: SET PTR->NEXT = START->NEXT
Step 6: SET START->NEXT->PREV = PTR
Step 7: FREE START
Step 8: SET START = PTR->NEXT

```

Figure 6.62 Algorithm to delete the first node

However, if there are nodes in the linked list, then we use a pointer variable PTR that is set to point to the first node of the list. For this, we initialize PTR with START that stores the address of the first node of the list. The while loop traverses through the list to reach the last node. Once we reach the last node, the NEXT pointer of PTR is set to contain the address of the node that succeeds START. Finally, START is made to point to the next node in the sequence and the memory occupied by the first node of the list is freed and returned to the free pool.

Deleting the Last Node from a Circular Doubly Linked List

Consider the circular doubly linked list shown in Fig. 6.63. Suppose we want to delete the last node from the linked list, then the following changes will be done in the linked list.

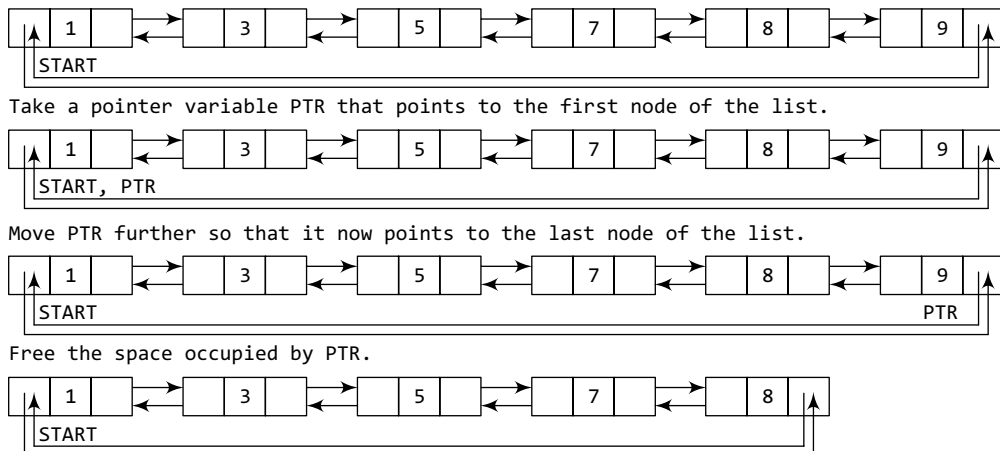


Figure 6.63 Deleting the last node from a circular doubly linked list

```

Step 1: IF START = NULL
        Write UNDERFLOW
        Go to Step 8
    [END OF IF]
Step 2: SET PTR = START
Step 3: Repeat Step 4 while PTR->NEXT != START
Step 4:     SET PTR = PTR->NEXT
    [END OF LOOP]
Step 5: SET PTR->PREV->NEXT = START
Step 6: SET START->PREV = PTR->PREV
Step 7: FREE PTR
Step 8: EXIT

```

Figure 6.64 Algorithm to delete the last node

Figure 6.64 shows the algorithm to delete the last node from a circular doubly linked list. In Step 2, we take a pointer variable PTR and initialize it with START. That is, PTR now points to the first node of the linked list. The while loop traverses through the list to reach the last node. Once we reach the last node, we can also access the second last node by taking its address from the PREV field of the last node. To delete the last node, we simply have to set the next field of the second last node to contain the address of START, so that it now becomes the (new) last node of the linked list. The memory of the previous last node is freed and returned to the free pool.

PROGRAMMING EXAMPLE

- Write a program to create a circular doubly linked list and perform insertions and deletions at the beginning and end of the list.

```

#include <stdio.h>
#include <conio.h>
#include <malloc.h>
struct node
{
    struct node *next;
    int data;
    struct node *prev;
};
struct node *start = NULL;
struct node *create_ll(struct node *);
struct node *display(struct node *);
struct node *insert_beg(struct node *);
struct node *insert_end(struct node *);
struct node *delete_beg(struct node *);
struct node *delete_end(struct node *);
struct node *delete_node(struct node *);
struct node *delete_list(struct node *);
int main()
{
    int option;
    clrscr();
    do
    {
        printf("\n\n *****MAIN MENU *****");
        printf("\n 1: Create a list");
        printf("\n 2: Display the list");
        printf("\n 3: Add a node at the beginning");
        printf("\n 4: Add a node at the end");
        printf("\n 5: Delete a node from the beginning");
        printf("\n 6: Delete a node from the end");
        printf("\n 7: Delete a given node");
        printf("\n 8: Delete the entire list");
        printf("\n 9: EXIT");
        printf("\n\n Enter your option : ");
    }

```

```
        scanf("%d", &option);
        switch(option)
        {
            case 1: start = create_ll(start);
                    printf("\n CIRCULAR DOUBLY LINKED LIST CREATED");
                    break;
            case 2: start = display(start);
                    break;
            case 3: start = insert_beg(start);
                    break;
            case 4: start = insert_end(start);
                    break;
            case 5: start = delete_beg(start);
                    break;
            case 6: start = delete_end(start);
                    break;
            case 7: start = delete_node(start);
                    break;
            case 8: start = delete_list(start);
                    printf("\n CIRCULAR DOUBLY LINKED LIST DELETED");
                    break;
        }
    }while(option != 9);
    getch();
    return 0;
}

struct node *create_ll(struct node *start)
{
    struct node *new_node, *ptr;
    int num;
    printf("\n Enter -1 to end");
    printf("\n Enter the data : ");
    scanf("%d", &num);
    while(num != -1)
    {
        if(start == NULL)
        {
            new_node = (struct node*)malloc(sizeof(struct node));
            new_node->prev = NULL;
            new_node->data = num;
            new_node->next = start;
            start = new_node;
        }
        else
        {
            new_node = (struct node*)malloc(sizeof(struct node));
            new_node->data = num;
            ptr = start;
            while(ptr->next != start)
                ptr = ptr->next;
            new_node->prev = ptr;
            ptr->next = new_node;
            new_node->next = start;
            start->prev = new_node;
        }
        printf("\n Enter the data : ");
        scanf("%d", &num);
    }
}
```

```

        return start;
    }
    struct node *display(struct node *start)
    {
        struct node *ptr;
        ptr = start;
        while(ptr->next != start)
        {
            printf("\t %d", ptr->data);
            ptr = ptr->next;
        }
        printf("\t %d", ptr->data);
        return start;
    }
    struct node *insert_beg(struct node *start)
    {
        struct node *new_node, *ptr;
        int num;
        printf("\n Enter the data : ");
        scanf("%d", &num);
        new_node = (struct node *)malloc(sizeof(struct node));
        new_node->data = num;
        ptr = start;
        while(ptr->next != start)
            ptr = ptr->next;
        new_node->prev = ptr;
        ptr->next = new_node;
        new_node->next = start;
        start->prev = new_node;
        start = new_node;
        return start;
    }
    struct node *insert_end(struct node *start)
    {
        struct node *ptr, *new_node;
        int num;
        printf("\n Enter the data : ");
        scanf("%d", &num);
        new_node = (struct node *)malloc(sizeof(struct node));
        new_node->data = num;
        ptr = start;
        while(ptr->next != start)
            ptr = ptr->next;
        ptr->next = new_node;
        new_node->prev = ptr;
        new_node->next = start;
        start->prev = new_node;
        return start;
    }
    struct node *delete_beg(struct node *start)
    {
        struct node *ptr;
        ptr = start;
        while(ptr->next != start)
            ptr = ptr->next;
        ptr->next = start->next;
        temp = start;
        start = start->next;
        start->prev = ptr;
        free(temp);
        return start;
    }

```

```

}
struct node *delete_end(struct node *start)
{
    struct node *ptr;
    ptr=start;
    while(ptr->next != start)
        ptr = ptr->next;
    ptr->prev->next = start;
    start->prev = ptr->prev;
    free(ptr);
    return start;
}
struct node *delete_node(struct node *start)
{
    struct node *ptr;
    int val;
    printf("\n Enter the value of the node which has to be deleted : ");
    scanf("%d", &val);
    ptr = start;
    if(ptr->data == val)
    {
        start = delete_beg(start);
        return start;
    }
    else
    {
        while(ptr->data != val)
            ptr = ptr->next;
        ptr->prev->next = ptr->next;
        ptr->next->prev = ptr->prev;
        free(ptr);
        return start;
    }
}
struct node *delete_list(struct node *start)
{
    struct node *ptr;
    ptr = start;
    while(ptr->next != start)
        start = delete_end(start);
    free(start);
    return start;
}

```

Output

```

*****MAIN MENU *****
1: Create a list
2: Display the list
-----
8: Delete the entire list
9: EXIT
Enter your option : 1
Enter -1 to end
Enter the data: 2
Enter the data: 3
Enter the data: 4
Enter the data: -1
CIRCULAR DOUBLY LINKED LIST CREATED
Enter your option : 8
CIRCULAR DOUBLY LINKED LIST DELETED
Enter your option : 9

```

6.6 HEADER LINKED LISTS

A header linked list is a special type of linked list which contains a header node at the beginning of the list. So, in a header linked list, **START** will not point to the first node of the list but **START** will contain the address of the header node. The following are the two variants of a header linked list:

- *Grounded header linked list* which stores **NULL** in the next field of the last node.
- *Circular header linked list* which stores the address of the header node in the next field of the last node. Here, the header node will denote the end of the list.

Look at Fig. 6.65 which shows both the types of header linked lists.

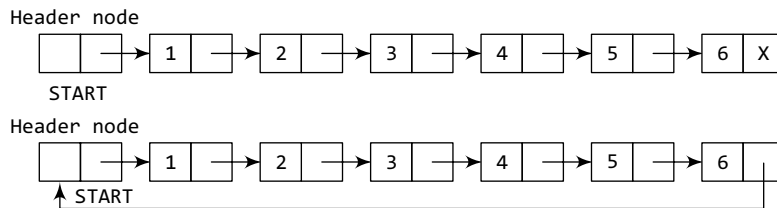


Figure 6.65 Header linked list

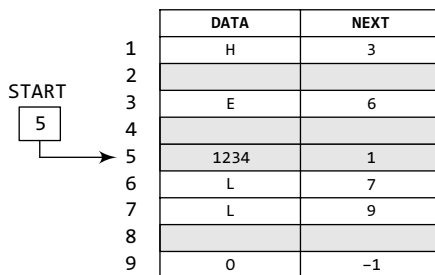


Figure 6.66 Memory representation of a header linked list

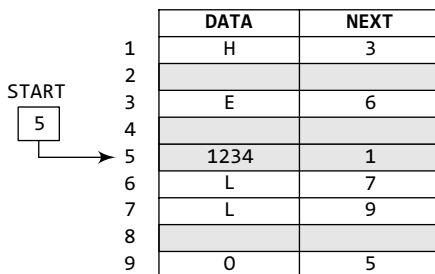


Figure 6.67 Memory representation of a circular header linked list

```

Step 1: SET PTR = START -> NEXT
Step 2: Repeat Steps 3 and 4 while PTR != START
Step 3:     Apply PROCESS to PTR -> DATA
Step 4:     SET PTR = PTR -> NEXT
          [END OF LOOP]
Step 5: EXIT

```

Figure 6.68 Algorithm to traverse a circular header linked list

As in other linked lists, if **START = NULL**, then this denotes an empty header linked list. Let us see how a grounded header linked list is stored in the memory. In a grounded header linked list, a node has two fields, **DATA** and **NEXT**. The **DATA** field will store the information part and the **NEXT** field will store the address of the node in sequence. Consider Fig. 6.66.

Note that **START** stores the address of the header node. The shaded row denotes a header node. The **NEXT** field of the header node stores the address of the first node of the list. This node stores **H**. The corresponding **NEXT** field stores the address of the next node, which is 3. So, we will look at address 3 to fetch the next data item.

Hence, we see that the first node can be accessed by writing **FIRST_NODE = START -> NEXT** and not by writing **START = FIRST_NODE**. This is because **START** points to the header node and the header node points to the first node of the header linked list.

Let us now see how a circular header linked list is stored in the memory. Look at Fig. 6.67.

Note that the last node in this case stores the address of the header node (instead of -1).

Hence, we see that the first node can be accessed by writing **FIRST_NODE = START -> NEXT** and not writing **START = FIRST_NODE**. This is because **START** points to the header node and the header node points to the first node of the header linked list.

Let us quickly look at Figs 6.68, 6.69, and 6.70 that show the algorithms to traverse a circular header linked list, insert a new node in it, and delete an existing node from it.

```

Step 1: IF AVAIL = NULL
        Write OVERFLOW
        Go to Step 10
    [END OF IF]
Step 2: SET NEW_NODE = AVAIL
Step 3: SET AVAIL = AVAIL->NEXT
Step 4: SET PTR = START->NEXT
Step 5: SET NEW_NODE->DATA = VAL
Step 6: Repeat Step 7 while PTR->DATA != NUM
Step 7:     SET PTR = PTR->NEXT
    [END OF LOOP]
Step 8: NEW_NODE->NEXT = PTR->NEXT
Step 9: SET PTR->NEXT = NEW_NODE
Step 10: EXIT

```

Figure 6.69 Algorithm to insert a new node in a circular header linked list

```

Step 1: SET PTR = START->NEXT
Step 2: Repeat Steps 3 and 4 while
        PTR->DATA != VAL
Step 3:     SET PREPTR = PTR
Step 4:     SET PTR = PTR->NEXT
    [END OF LOOP]
Step 5: SET PREPTR->NEXT = PTR->NEXT
Step 6: FREE PTR
Step 7: EXIT

```

Figure 6.70 Algorithm to delete a node from a circular header linked list

After discussing linked lists in such detail, these algorithms are self-explanatory. There is actually just one small difference between these algorithms and the algorithms that we have discussed earlier. Like we have a header list and a circular header list, we also have a two-way (doubly) header list and a circular two-way (doubly) header list. The algorithms to perform all the basic operations will be exactly the same except that the first node will be accessed by writing `START->NEXT` instead of `START`.

PROGRAMMING EXAMPLE

5. Write a program to implement a header linked list.

```

#include <stdio.h>
#include <conio.h>
#include <malloc.h>
struct node
{
    int data;
    struct node *next;
};
struct node *start = NULL;
struct node *create_hll(struct node *);
struct node *display(struct node *);
int main()
{
    int option;
    clrscr();
    do
    {
        printf("\n\n *****MAIN MENU *****");
        printf("\n 1: Create a list");
        printf("\n 2: Display the list");
        printf("\n 3: EXIT");
        printf("\n Enter your option : ");
        scanf("%d", &option);
        switch(option)
        {
            case 1: start = create_hll(start);
                    printf("\n HEADER LINKED LIST CREATED");
                    break;

```

```

        case 2: start = display(start);
                break;
        }
    }while(option !=3);
    getch();
    return 0;
}
struct node *create_hll(struct node *start)
{
    struct node *new_node, *ptr;
    int num;
    printf("\n Enter -1 to end");
    printf("\n Enter the data : ");
    scanf("%d", &num);
    while(num!=-1)
    {
        new_node = (struct node*)malloc(sizeof(struct node));
        new_node->data=num;
        new_node->next=NULL;
        if(start==NULL)
        {
            start = (struct node*)malloc(sizeof(struct node));
            start->next=new_node;
        }
        else
        {
            ptr=start;
            while(ptr->next!=NULL)
                ptr=ptr->next;
            ptr->next=new_node;
        }
        printf("\n Enter the data : ");
        scanf("%d", &num);
    }
    return start;
}
struct node *display(struct node *start)
{
    struct node *ptr;
    ptr=start;
    while(ptr!=NULL)
    {
        printf("\t %d", ptr->data);
        ptr = ptr->next;
    }
    return start;
}

```

Output

```

*****MAIN MENU *****
1: Create a list
2: Display the list
3: EXIT
Enter your option : 1
Enter -1 to end
Enter the data: 1
Enter the data: 2
Enter the data: 4
Enter the data: -1
HEADER LINKED LIST CREATED
Enter your option : 3

```


6.7 MULTI-LINKED LISTS

In a multi-linked list, each node can have n number of pointers to other nodes. A doubly linked list is a special case of multi-linked lists. However, unlike doubly linked lists, nodes in a multi-linked list may or may not have inverses for each pointer. We can differentiate a doubly linked list from a multi-linked list in two ways:

- A doubly linked list has exactly two pointers. One pointer points to the previous node and the other points to the next node. But a node in the multi-linked list can have any number of pointers.
- In a doubly linked list, pointers are exact inverses of each other, i.e., for every pointer which points to a previous node there is a pointer which points to the next node. This is not true for a multi-linked list.

Multi-linked lists are generally used to organize multiple orders of one set of elements. For example, if we have a linked list that stores name and marks obtained by students in a class, then we can organize the nodes of the list in two ways:

- Organize the nodes alphabetically (according to the name)
- Organize the nodes according to decreasing order of marks so that the information of student who got highest marks comes before other students.

Figure 6.71 shows a multi-linked list in which students' nodes are organized by both the aforementioned ways.

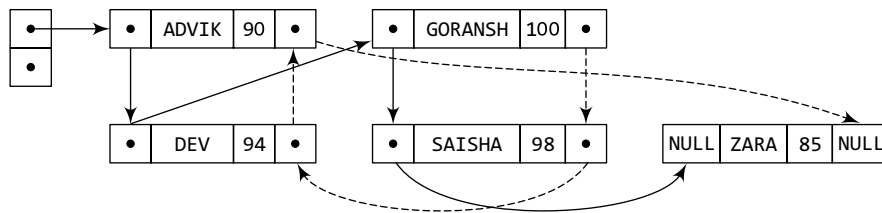


Figure 6.71 Multi-linked list that stores names alphabetically as well as according to decreasing order of marks

A new node can be inserted in a multi-linked list in the same way as it is done for a doubly linked list.

Note In multi-linked lists, we can have inverses of each pointer as in a doubly linked list. But for that we must have four pointers in a single node.

y \ x	x		
	0	1	2
0	0	25	0
1	0	0	0
2	17	0	5
3	19	0	0

Figure 6.72 Sparse matrix

Multi-linked lists are also used to store sparse matrices. In Chapter 3 we have read about sparse matrices. Such matrices have very few non-zero values stored and most of the entries are zero. Sparse matrices are very common in engineering applications. If we use a normal array to store such matrices, we will end up wasting a lot of space. Therefore, a better solution is to represent these matrices using multi-linked lists.

The sparse matrix shown in Fig. 6.72 can be represented using a linked list for every row and column. Since a value is in exactly one row and one column, it will appear in both lists exactly once. A node in the multi-linked will have four parts. First stores the data, second stores a pointer to the next node in the row, third stores a pointer to the next node in the column, and the fourth stores the coordinates or the row and column number in which the data appears in

the matrix. However, as in case of doubly linked lists, we can also have a corresponding inverse pointer for every pointer in the multi-linked list representation of a sparse matrix.

Note When a non-zero value in the sparse matrix is set to zero, the corresponding node in the multi-linked list must be deleted.

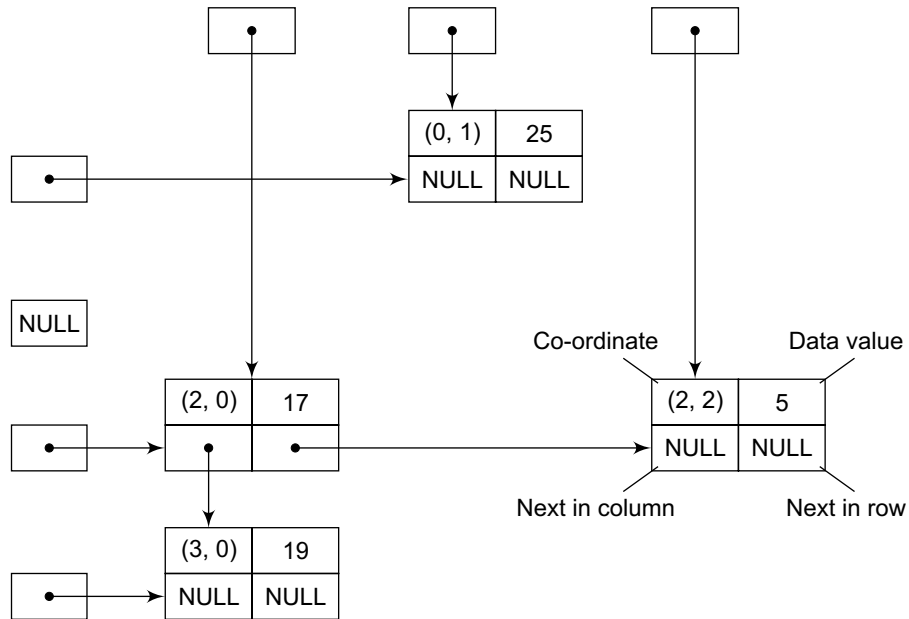


Figure 6.73 Multi-linked representation of sparse matrix shown in Fig. 6.72

6.8 APPLICATIONS OF LINKED LISTS

Linked lists can be used to represent polynomials and the different operations that can be performed on them. In this section, we will see how polynomials are represented in the memory using linked lists.

6.8.1 Polynomial Representation

Let us see how a polynomial is represented in the memory using a linked list. Consider a polynomial $6x^3 + 9x^2 + 7x + 1$. Every individual term in a polynomial consists of two parts, a coefficient and a power. Here, 6, 9, 7, and 1 are the coefficients of the terms that have 3, 2, 1, and 0 as their powers respectively.

Every term of a polynomial can be represented as a node of the linked list. Figure 6.74 shows the linked representation of the terms of the above polynomial.

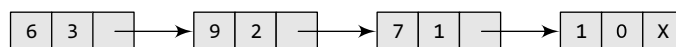


Figure 6.74 Linked representation of a polynomial

Now that we know how polynomials are represented using nodes of a linked list, let us write a program to perform operations on polynomials.

PROGRAMMING EXAMPLE

6. Write a program to store a polynomial using linked list. Also, perform addition and subtraction on two polynomials.

```
#include <stdio.h>
#include <conio.h>
#include <malloc.h>
struct node
{
    int num;
    int coeff;
    struct node *next;
};
struct node *start1 = NULL;
struct node *start2 = NULL;
struct node *start3 = NULL;
struct node *start4 = NULL;
struct node *last3 = NULL;
struct node *create_poly(struct node *);
struct node *display_poly(struct node *);
struct node *add_poly(struct node *, struct node *, struct node *);
struct node *sub_poly(struct node *, struct node *, struct node *);
struct node *add_node(struct node *, int, int);
int main()
{
    int option;
    clrscr();
    do
    {
        printf("\n***** MAIN MENU *****");
        printf("\n 1. Enter the first polynomial");
        printf("\n 2. Display the first polynomial");
        printf("\n 3. Enter the second polynomial");
        printf("\n 4. Display the second polynomial");
        printf("\n 5. Add the polynomials");
        printf("\n 6. Display the result");
        printf("\n 7. Subtract the polynomials");
        printf("\n 8. Display the result");
        printf("\n 9. EXIT");
        printf("\n\n Enter your option : ");
        scanf("%d", &option);
        switch(option)
        {
            case 1: start1 = create_poly(start1);
                    break;
            case 2: start1 = display_poly(start1);
                    break;
            case 3: start2 = create_poly(start2);
                    break;
            case 4: start2 = display_poly(start2);
                    break;
            case 5: start3 = add_poly(start1, start2, start3);
                    break;
            case 6: start3 = display_poly(start3);
                    break;
            case 7: start4 = sub_poly(start1, start2, start4);
                    break;
            case 8: start4 = display_poly(start4);
```

```

        break;
    }
    }while(option!=9);
    getch();
    return 0;
}
struct node *create_poly(struct node *start)
{
    struct node *new_node, *ptr;
    int n, c;
    printf("\n Enter the number : ");
    scanf("%d", &n);
    printf("\t Enter its coefficient : ");
    scanf("%d", &c);
    while(n != -1)
    {
        if(start==NULL)
        {
            new_node = (struct node *)malloc(sizeof(struct node));
            new_node->num = n;
            new_node->coeff = c;
            new_node->next = NULL;
            start = new_node;
        }
        else
        {
            ptr = start;
            while(ptr->next != NULL)
                ptr = ptr->next;
            new_node = (struct node *)malloc(sizeof(struct node));
            new_node->num = n;
            new_node->coeff = c;
            new_node->next = NULL;
            ptr->next = new_node;
        }
        printf("\n Enter the number : ");
        scanf("%d", &n);
        if(n == -1)
            break;
        printf("\t Enter its coefficient : ");
        scanf("%d", &c);
    }
    return start;
}
struct node *display_poly(struct node *start)
{
    struct node *ptr;
    ptr = start;
    while(ptr != NULL)
    {
        printf("\n%d x %d\t", ptr->num, ptr->coeff);
        ptr = ptr->next;
    }
    return start;
}
struct node *add_poly(struct node *start1, struct node *start2, struct node *start3)
{
    struct node *ptr1, *ptr2;
    int sum_num, c;

```

```

ptr1 = start1, ptr2 = start2;
while(ptr1 != NULL && ptr2 != NULL)
{
    if(ptr1->coeff == ptr2->coeff)
    {
        sum_num = ptr1->num + ptr2->num;
        start3 = add_node(start3, sum_num, ptr1->coeff);
        ptr1 = ptr1->next;
        ptr2 = ptr2->next;
    }
    else if(ptr1->coeff > ptr2->coeff)
    {
        start3 = add_node(start3, ptr1->num, ptr1->coeff);
        ptr1 = ptr1->next;
    }
    else if(ptr1->coeff < ptr2->coeff)
    {
        start3 = add_node(start3, ptr2->num, ptr2->coeff);
        ptr2 = ptr2->next;
    }
}
if(ptr1 == NULL)
{
    while(ptr2 != NULL)
    {
        start3 = add_node(start3, ptr2->num, ptr2->coeff);
        ptr2 = ptr2->next;
    }
}
if(ptr2 == NULL)
{
    while(ptr1 != NULL)
    {
        start3 = add_node(start3, ptr1->num, ptr1->coeff);
        ptr1 = ptr1->next;
    }
}
return start3;
}
struct node *sub_poly(struct node *start1, struct node *start2, struct node *start4)
{
    struct node *ptr1, *ptr2;
    int sub_num, c;
    ptr1 = start1, ptr2 = start2;
    do
    {
        if(ptr1->coeff == ptr2->coeff)
        {
            sub_num = ptr1->num - ptr2->num;
            start4 = add_node(start4, sub_num, ptr1->coeff);
            ptr1 = ptr1->next;
            ptr2 = ptr2->next;
        }
        else if(ptr1->coeff > ptr2->coeff)
        {
            start4 = add_node(start4, ptr1->num, ptr1->coeff);
            ptr1 = ptr1->next;
        }
        else if(ptr1->coeff < ptr2->coeff)

```

```

        {
            start4 = add_node(start4, ptr2->num, ptr2->coeff);
            ptr2 = ptr2->next;
        }
    }while(ptr1 != NULL || ptr2 != NULL);
    if(ptr1 == NULL)
    {
        while(ptr2 != NULL)
        {
            start4 = add_node(start4, ptr2->num, ptr2->coeff);
            ptr2 = ptr2->next;
        }
    }
    if(ptr2 == NULL)
    {
        while(ptr1 != NULL)
        {
            start4 = add_node(start4, ptr1->num, ptr1->coeff);
            ptr1 = ptr1->next;
        }
    }
    return start4;
}

struct node *add_node(struct node *start, int n, int c)
{
    struct node *ptr, *new_node;
    if(start == NULL)
    {
        new_node = (struct node *)malloc(sizeof(struct node));
        new_node->num = n;
        new_node->coeff = c;
        new_node->next = NULL;
        start = new_node;
    }
    else
    {
        ptr = start;
        while(ptr->next != NULL)
            ptr = ptr->next;
        new_node = (struct node *)malloc(sizeof(struct node));
        new_node->num = n;
        new_node->coeff = c;
        new_node->next = NULL;
        ptr->next = new_node;
    }
    return start;
}

```

Output

```

***** MAIN MENU *****
1. Enter the first polynomial
2. Display the first polynomial
-----
9. EXIT
Enter your option : 1
Enter the number : 6      Enter its coefficient : 2
Enter the number : 5      Enter its coefficient : 1
Enter the number : -1
Enter your option : 2
6 x 2    5 x 1
Enter your option : 9

```

POINTS TO REMEMBER

- A linked list is a linear collection of data elements called as nodes in which linear representation is given by links from one node to another.
- Linked list is a data structure which can be used to implement other data structures such as stacks, queues, and their variations.
- Before we insert a new node in linked lists, we need to check for OVERFLOW condition, which occurs when no free memory cell is present in the system.
- Before we delete a node from a linked list, we must first check for UNDERFLOW condition which occurs when we try to delete a node from a linked list that is empty.
- When we delete a node from a linked list, we have to actually free the memory occupied by that node. The memory is returned back to the free pool so that it can be used to store other programs and data.
- In a circular linked list, the last node contains a pointer to the first node of the list. While traversing a circular linked list, we can begin at any node and traverse the list in any direction forward or backward until we reach the same node where we had started.
- A doubly linked list or a two-way linked list is a linked list which contains a pointer to the next as well as the previous node in the sequence. Therefore, it consists of three parts—data, a pointer to the next node, and a pointer to the previous node.
- The PREV field of the first node and the NEXT field of the last node contain NULL. This enables to traverse the list in the backward direction as well.
- Thus, a doubly linked list calls for more space per node and more expensive basic operations. However, a doubly linked list provides the ease to manipulate the elements of the list as it maintains pointers to nodes in both the directions (forward and backward). The main advantage of using a doubly linked list is that it makes search operation twice as efficient.
- A circular doubly linked list or a circular two-way linked list is a more complex type of linked list which contains a pointer to the next as well as previous node in the sequence. The difference between a doubly linked and a circular doubly linked list is that the circular doubly linked list does not contain NULL in the previous field of the first node and the next field of the last node. Rather, the next field of the last node stores the address of the first node of the list. Similarly, the previous field of the first field stores the address of the last node.
- A header linked list is a special type of linked list which contains a header node at the beginning of the list. So, in a header linked list START will not point to the first node of the list but START will contain the address of the header node.
- Multi-linked lists are generally used to organize multiple orders of one set of elements. In a multi-linked list, each node can have n number of pointers to other nodes.

EXERCISES

Review Questions

1. Make a comparison between a linked list and a linear array. Which one will you prefer to use and when?
2. Why is a doubly linked list more useful than a singly linked list?
3. Give the advantages and uses of a circular linked list.
4. Specify the use of a header node in a header linked list.
5. Give the linked representation of the following polynomial:

$$7x^3y^2 - 8x^2y + 3xy + 11x - 4$$
6. Explain the difference between a circular linked list and a singly linked list.
7. Form a linked list to store students' details.
8. Use the linked list of the above question to insert the record of a new student in the list.
9. Delete the record of a student with a specified roll number from the list maintained in Question 7.
10. Given a linked list that contains English alphabet. The characters may be in upper case or in lower case. Create two linked lists—one which stores upper case characters and the other that stores lower case characters.

11. Create a linked list which stores names of the employees. Then sort these names and re-display the contents of the linked list.

Programming Exercises

1. Write a program that removes all nodes that have duplicate information.
2. Write a program to print the total number of occurrences of a given item in the linked list.
3. Write a program to multiply every element of the linked list with 10.
4. Write a program to print the number of non-zero elements in the list.
5. Write a program that prints whether the given linked list is sorted (in ascending order) or not.
6. Write a program that copies a circular linked list.
7. Write a program to merge two linked lists.
8. Write a program to sort the values stored in a doubly circular linked list.
9. Write a program to merge two sorted linked lists. The resultant list must also be sorted.
10. Write a program to delete the first, last, and middle node of a header linked list.
11. Write a program to create a linked list from an already given list. The new linked list must contain every alternate element of the existing linked list.
12. Write a program to concatenate two doubly linked lists.
13. Write a program to delete the first element of a doubly linked list. Add this node as the last node of the list.
14. Write a program to
 - (a) Delete the first occurrence of a given character in a linked list
 - (b) Delete the last occurrence of a given character
 - (c) Delete all the occurrences of a given character
15. Write a program to reverse a linked list using recursion.
16. Write a program to input an n digit number. Now, break this number into its individual digits and then store every single digit in a separate node thereby forming a linked list. For example, if you enter 12345, then there will 5 nodes in the list containing nodes with values 1, 2, 3, 4, 5.
17. Write a program to add the values of the nodes of a linked list and then calculate the mean.
18. Write a program that prints minimum and maximum values in a linked list that stores integer values.
19. Write a program to interchange the value of the first element with the last element, second element with second last element, so on and so forth of a doubly linked list.
20. Write a program to make the first element of singly linked list as the last element of the list.
21. Write a program to count the number of occurrences of a given value in a linked list.
22. Write a program that adds 10 to the values stored in the nodes of a doubly linked list.
23. Write a program to form a linked list of floating point numbers. Display the sum and mean of these numbers.
24. Write a program to delete the k^{th} node from a linked list.
25. Write a program to perform deletions in all the cases of a circular header linked list.
26. Write a program to multiply a polynomial with a given number.
27. Write a program to count the number of non-zero values in a circular linked list.
28. Write a program to create a linked list which stores the details of employees in a department. Read and print the information stored in the list.
29. Use the linked list of Question 28 so that it displays the record of a given employee only.
30. Use the linked list of Question 28 and insert information about a new employee.
31. Use the linked list of Question 28 and delete information about an existing employee.
32. Write a program to move a middle node of a doubly link list to the top of the list.
33. Write a program to create a singly linked list and reverse the list by interchanging the links and not the data.
34. Write a program that prints the n th element from the end of a linked list in a single pass.
35. Write a program that creates a singly linked list. Use a function `isSorted` that returns 1 if the list is sorted and 0 otherwise.
36. Write a program to interchange the k th and the $(k+1)$ th node of a circular doubly linked list.
37. Write a program to create a header linked list.

38. Write a program to delete a node from a circular header linked list.
39. Write a program to delete all nodes from a header linked list that has negative values in its data part.

Multiple-choice Questions

1. A linked list is a
 - (a) Random access structure
 - (b) Sequential access structure
 - (c) Both
 - (d) None of these
2. An array is a
 - (a) Random access structure
 - (b) Sequential access structure
 - (c) Both
 - (d) None of these
3. Linked list is used to implement data structures like
 - (a) Stacks
 - (b) Queues
 - (c) Trees
 - (d) All of these
4. Which type of linked list contains a pointer to the next as well as the previous node in the sequence?
 - (a) Singly linked list
 - (b) Circular linked list
 - (c) Doubly linked list
 - (d) All of these
5. Which type of linked list does not store NULL in next field?
 - (a) Singly linked list
 - (b) Circular linked list
 - (c) Doubly linked list
 - (d) All of these
6. Which type of linked list stores the address of the header node in the next field of the last node?
 - (a) Singly linked list
 - (b) Circular linked list
 - (c) Doubly linked list
 - (d) Circular header linked list
7. Which type of linked list can have four pointers per node?
 - (a) Circular doubly linked list
 - (b) Multi-linked list
 - (c) Header linked list
 - (d) Doubly linked list

True or False

1. A linked list is a linear collection of data elements.
2. A linked list can grow and shrink during run time.

3. A node in a linked list can point to only one node at a time.
4. A node in a singly linked list can reference the previous node.
5. A linked list can store only integer values.
6. Linked list is a random access structure.
7. Deleting a node from a doubly linked list is easier than deleting it from a singly linked list.
8. Every node in a linked list contains an integer part and a pointer.
9. START stores the address of the first node in the list.
10. Underflow is a condition that occurs when we try to delete a node from a linked list that is empty.

Fill in the Blanks

1. _____ is used to store the address of the first free memory location.
2. The complexity to insert a node at the beginning of the linked list is _____.
3. The complexity to delete a node from the end of the linked list is _____.
4. Inserting a node at the beginning of the doubly linked list needs to modify _____ pointers.
5. Inserting a node in the middle of the singly linked list needs to modify _____ pointers.
6. Inserting a node at the end of the circular linked list needs to modify _____ pointers.
7. Inserting a node at the beginning of the circular doubly linked list needs to modify _____ pointers.
8. Deleting a node from the beginning of the singly linked list needs to modify _____ pointers.
9. Deleting a node from the middle of the doubly linked list needs to modify _____ pointers.
10. Deleting a node from the end of a circular linked list needs to modify _____ pointers.
11. Each element in a linked list is known as a _____.
12. First node in the linked list is called the _____.
13. Data elements in a linked list are known as _____.
14. Overflow occurs when _____.
15. In a circular linked list, the last node contains a pointer to the _____ node of the list.